drones

MDPI

*Article*

# H-CoRE: A Cooperative Framework for Heterogeneous Multi-Robot Exploration and Inspection

**Simone D'Angelo [1,*], Francesca Pagano [1], Riccardo Caccavale [1], Vincenzo Scognamiglio [2], Alessandro De Crescenzo [3], Pasquale Merone [1], Stefano Ciaravino [1], Alberto Finzi [1] and Vincenzo Lippiello [1,3]**

[1] PRISMA Lab, Department of Engineering and Information Technology, University of Naples Federico II, 80125 Naples, Italy
[2] KRUSH Labs B.V., High Tech Campus 32, 5656 AE Eindhoven, The Netherlands
[3] Neabotics s.r.l., Via G. Marconi 15, 80125 Naples, Italy
* Correspondence: simone.dangelo@unina.it

## Highlights

**What are the main findings?**

- The H-CoRE framework enables autonomous exploration and inspection with a heterogeneous robotic team in GNSS-denied environments.
- The proposed architecture allows flexible task decomposition, dynamic scheduling, and real-time coordination.

**What is the implication of the main findings?**

- As H-CoRE is open source, it can provide a modular, flexible, and scalable software foundation for advancing research and deployment of heterogeneous multi-robot teams.
- The framework demonstrates a practical path toward robust and scalable autonomous missions, enabling integration of diverse robotic platforms with minimal inter-agent information exchange.

## Abstract

This paper presents the H-CoRE (Heterogeneous Cooperative Multi-Robot Execution) framework designed to enable autonomous multi-robot operations in GNSS-denied environments. Built on an ROS 2-based architecture, H-CoRE enables collaborative, structured task execution through standardized software stacks. Each robot's stack combines a high-level executive system with an agent-specific motion layer and leverages multi-sensor fusion for localization and mapping. The framework is inherently reconfigurable, allowing individual agents to operate autonomously or as part of a multi-robot team for collaborative missions. In the considered scenario, the system integrates aerial and ground vehicles, a fixed pan–tilt–zoom camera, and a human supervisory interface within a unified, modular infrastructure. The proposed system has been deployed in indoor, GNSS-denied environments, demonstrating autonomous navigation, cooperative area coverage, and real-time information sharing across multiple agents. Experimental results confirm the effectiveness of H-CoRE in maintaining general awareness and mission continuity, paving the way for future applications in search-and-rescue, inspection, and exploration tasks.

**Keywords:** heterogeneous multi-robot systems; cooperative exploration; UAV–UGV collaboration; GNSS-denied environments; task scheduling

Check for updates

Academic Editor: Andrey V. Savkin

Received: 22 January 2026
Revised: 16 March 2026
Accepted: 20 March 2026
Published: 25 March 2026

**Copyright:** © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

## 1. Introduction

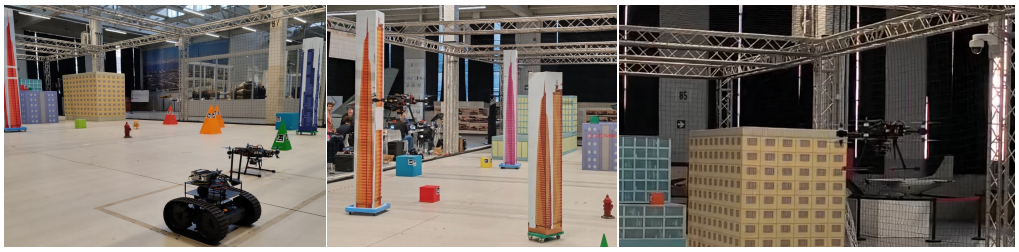
Autonomous robotic systems are being increasingly deployed in complex and unstructured environments where Global Navigation Satellite System (GNSS) signals are unavailable or unreliable. Such conditions often occur for indoor facilities [1], subterranean infrastructures [2], and post-disaster areas [3], where robots must rely exclusively on onboard sensing and local decision-making to navigate the environment and execute their mission [4,5]. In these contexts, achieving robust autonomy remains a major challenge that often requires the integration of multiple sensing modalities, effective communication among agents, and advanced coordination strategies [6,7].

Heterogeneous multi-robot systems offer a promising solution to these challenges by exploiting the complementary capabilities of different platforms. Unmanned Aerial Vehicles (UAVs), also known as drones, bring high mobility, rapid coverage, and a broad view of the environment. In contrast, Unmanned Ground Vehicles (UGVs) facilitate detailed inspection, long-term operation, and interaction with the environment. When operating cooperatively, these agents can jointly build maps, share situational awareness, and distribute exploration tasks, achieving levels of efficiency and robustness that exceed those of individual platforms. Furthermore, integrating human operators and fixed sensing units, such as pan–tilt–zoom (PTZ) cameras, can enhance mission supervision and extend the perceptual horizon of the robotic network [8].

In this context, the design of software frameworks plays a crucial role in advancing heterogeneous multi-robot systems research and real-world deployment. As underlined by Iñigo-Blasco et al. [9], such frameworks help manage the inherent complexity of multi-agent robotics by providing standardized programming models, modular architectures, and reusable components. Effective software infrastructures must also support distributed computation, efficient communication mechanisms, and offer the flexibility needed to accommodate diverse platforms and sensing payloads [7,10]. Over the past decade, several frameworks have been proposed to facilitate multi-robot exploration and cooperative perception [11–16]. However, many existing solutions remain tailored to homogeneous teams or depend on centralized control architectures, limiting scalability and robustness.

These limitations have also emerged in applied research contexts such as the Leonardo Drone Contest [17], an industrial competition which has progressively evolved from single-robot autonomy toward heterogeneous multi-robot teams operating in GNSS-denied environments. Early editions of the competition led to the development of fully autonomous UAV systems for perception, navigation, and mission execution, as documented in several works [18–22]. However, as the competition scenarios became more complex and introduced UAV–UGV-PTZ camera teaming (see Figure 1), the deployed solutions had to address the additional challenges posed by cooperation among heterogeneous robots and the execution of missions in realistic environments.

Motivated by these needs, this paper presents the H-CoRE (*Heterogeneous Cooperative Multi-Robot Execution*) framework for multi-agent exploration and inspection in GNSS-denied environments. The proposed architecture distinguishes itself from prior work through the use of cognitive-based executive systems to execute structured tasks and enable cooperation among heterogeneous robotic platforms, namely UAVs, UGVs, and PTZ cameras. To the best of our knowledge, this form of heterogeneous teaming has not been addressed in the existing literature. Moreover, to deploy this approach in the context of exploration and inspection, dedicated motion stacks/interfaces were developed for each agent, enabling a range of domain-specific motion primitives.



(**a**) UAV and UGV agents. (**b**) UAV among obstacles. (**c**) UAV and PTZ camera.

**Figure 1.** Snapshots from indoor exploration in the Leonardo Drone Contest arena.

H-CoRE implements a decentralized task scheduling and execution policy for each agent, assuming that mission and task allocation are given, which is consistent with human-assigned missions. The proposed architecture is general and reconfigurable, supporting deployment across diverse robots through class-specific modules that simplify adaptation to each platform's operational domain. Moreover, each robotic agent can operate independently using its dedicated motion stack or collaborate with others, sharing minimal task information in real time. This design enables H-CoRE to function both as an individual navigation framework and as a cooperative exploration and inspection system. These features foster scalability by enabling seamless system-wide reconfiguration, allowing agents and tasks to be added or removed without requiring modifications to the execution policies of the remaining agents. Thus, the main contributions are the following.

- A decentralized, human-supervised multi-robot architecture based on ROS 2, designed to be flexible, scalable, and modular.
- Integration of a cognitive-based executive system for flexible coordination of heterogeneous robots within a cooperative mission framework.
- Integration of dedicated motion stacks for UAVs, UGVs, and PTZ camera agents. These stacks are designed to exploit the specific capabilities of each platform and provide a set of motion primitives to support exploration and inspection tasks.
- The complete framework has been released as open source to the community https://github.com/Prisma-Drone-Team/H-CoRE (accessed on 1 January 2026) with packages and guidelines.
- H-CoRE is finally validated through experimental tests involving exploration and inspection missions, relying exclusively on onboard localization.

The rest of this paper is structured as follows. Section 2 reviews the related works underlining our contribution. Section 3 presents the overall architecture of H-CoRE, describing its main components. The modules of the individual heterogeneous agents are described in Section 4. Section 5 showcases the experimental results. Finally, Section 6 provides concluding remarks and outlines future research directions.

## 2. Related Works

Despite major advancements in field robotics over the past decade, the deployment of autonomous robots in unknown, unstructured environments remains challenging, and several frameworks have been proposed to support the integration of perception, control, and communication pipelines. This is particularly true in the UAV domain, where operational complexity has driven the development of dedicated solutions to address single-drone field deployment. Examples include BEAST [23], a framework enabling UAV beyond-visual-line-of-sight operations in real-world conditions but relying on GNSS, and LUCAS [24], a lightweight system for autonomous indoor inspection. UAV autonomous exploration and target detection in GNSS-denied environments is also addressed in [15],

while a related framework tackles both exploration and visual inspection of industrial facilities [25]. Ref. [21] presents an ROS–PX4-based framework for autonomous UAV navigation, deployed in the 2022 Leonardo Drone Contest, and describes the hardware and software solutions. Additionally, refs. [18,20] describe the drone architecture used in the same contest, including the integration of a human-teleoperated PTZ camera to support UAV exploration and inspection. Bianchi et al. [22] further propose a distributed ROS 2-based architecture that orchestrates perception, exploration, and navigation modules through a finite-state machine. However, these works remain focused on single-UAV systems, whereas multi-robot deployment introduces additional complexity, such as coordination, shared perception, and communication.

A multi-UAV framework for cooperative inspection of bridges is presented in [11], formulating the task as a composition of spatial–temporal requirements expressed through signal temporal logic. While validated in real-world conditions, this solution relies on precise GNSS and a centralized motion planning module, limiting the system's robustness and scalability. Another multi-drone framework is the MRS UAV system [26], which has significantly contributed to the research community thanks to its modular architecture and open-source implementation. This ecosystem has served as the basis for several applications, including the forest-mapping solution proposed in [12], and the multi-UAV system employed in the virtual DARPA subterranean search-and-rescue challenge [2].

In the context of the DARPA competition, other works [27–29] have further advanced the deployment of heterogeneous robotic teams under high-level supervision from a single human operator, demonstrating the effectiveness of multi-domain mobile agents in perceptually degraded environments. Notable examples include the CERBERUS [29] and Team CSIRO Data61 [27] systems. CERBERUS combines legged and flying robots to achieve resilient autonomy through unified exploration path planning and robot-specific, local motion planning. Team CSIRO Data61, in turn, integrates homogeneous sensor payloads across legged, wheeled, and aerial platforms, maintaining consistent perception and mapping via a centralized SLAM server. Although advanced, these frameworks were not released as open source, limiting their direct reusability. In addition to the DARPA challenge, other applications have leveraged heterogeneous robots' complementary sensing and actuation capabilities within different mission scenarios. A system combining UAVs and a UGV for the MBZIRC 2020 Challenge exploited LiDAR-based mapping and Behavior Tree mission execution to perform autonomous fire detection and suppression under communication constraints [30]. In a marine environment, multiple UAVs and an unmanned surface vehicle with a robotic arm were deployed to jointly perform autonomous retrieval of large floating objects using non-prehensile manipulation strategies [31].

Rather than addressing specific applicative scenarios, some frameworks propose general-purpose multi-robot frameworks for cooperative navigation, implementing target assignment, distributed control, and swarm behaviours [32,33]. However, these solutions are deployed exclusively in laboratory conditions using an external localization system and on homogeneous robotic teams. Similarly, the open-source ROS 2-based framework proposed for industrial automation in [13] relies on external overhead cameras and ArUco markers to localize ground robots, restricting deployment to controlled indoor environments.

The literature review, as summarized in Table 1, highlights that H-CoRE is the only architecture addressing the teaming of wheeled, aerial, and fixed-base robots. Moreover, to the best of our knowledge, it is the only fully open-source framework developed entirely in ROS 2 that enables onboard localization in GNSS-denied environments. This facilitates the integration of state-of-the-art software modules while exploiting native ROS 2 capabilities, such as Data Distribution Service (DDS) Quality-of-Service (QoS) policies for sensor streams. Consequently, it offers a more flexible and robust foundation for multi-robot operations in challenging scenarios, if compared to other ROS 2-based frameworks [13,33].

**Table 1.** High-level comparison between H-CoRE and state-of-the-art robotic frameworks.

| Ref. | Multi-Robot | Agent Type(s) | Onboard Localization | Middleware | Application | Open Source |
|---|---|---|---|---|---|---|
| H-CoRE | ✓ | UAV, UGV, PTZ camera | ✓ | ROS 2 | Exploration and inspection | ✓ |
| [23] | ✗ | UAV | ✗ | ROS | BVLS operations | ✗ |
| [24] | ✗ | UAV | ✓ | ROS/ROS 2 | Indoor inspection | ✓ |
| [15] | ✗ | UAV | ✓ | ROS | Navigation and target detection | ✓ |
| [25] | ✗ | UAV | ✓ | ROS | Visual inspection | ✗ |
| [21] | ✗ | UAV | ✓ | ROS | Leonardo Drone Contest | ✗ |
| [22] | ✗ | UAV | ✓ | ROS2 | Exploration and inspection | ✗ |
| [18,20] | ✗ | UAV, PTZ camera | ✓ | ROS | Leonardo Drone Contest | ✗ |
| [12] | ✓ | UAV | ✓ | ROS | Forest mapping | ✓ |
| [11] | ✓ | UAV | ✗ | ROS | Bridge inspection | ✓ |
| [30] | ✓ | UAVs, UGV | ✓ | ROS | Firefighting | ✓ |
| [26] | ✓ | UAVs | ✓ | ROS | General purpose | ✓ |
| [31] | ✓ | UAV, USV | ✓ | ROS | Maritime object retrieval | ✗ |
| [16] | ✓ | UAV, UGV | ✓ | ROS | Exploration | ✗ |
| [32] | ✓ | UGVs | ✗ | Khepera IV | Multi-robot navigation | ✗ |
| [2] | ✓ | UAVs | ✓ | ROS | Subterranean search and rescue | partially |
| [13] | ✓ | UGV, cameras | ✗ | ROS 2 | Industrial automation | ✓ |
| [33] | ✓ | UGVs | ✗ | ROS 2 | Multi-robot navigation and control | ✓ |

Moreover, Table 2 summarizes additional architectural aspects and provides a targeted comparison between H-CoRE and representative multi-robot frameworks from the literature. The analysis focuses on the capability of the systems to execute non-atomic tasks (*structured task execution and management*); the type of communicated messages in terms of dense or sparse data (*information sharing*); the ways in which the user can supervise and interact with the system (*human-in-the-loop management*); the capability of handling task-level failures (*task-level failure management*). In particular, H-CoRE is built upon a distributed, cognitive-based executive system, supports online replanning at both the task and motion levels, and maintains limited information exchange by relying solely on sparse data. The framework extends the authors' earlier work [16], not only incorporating the PTZ camera as a new robot class but also extending the overall system capabilities, such as enabling coordinated task execution and supporting HITL management.

This comparison highlights how H-CoRE differs from existing work in terms of architectural choices, team composition, system capabilities, and the availability of freely accessible resources.

**Table 2.** Comparison between the proposed H-CoRE framework and representative state-of-the-art multi-robot systems, focusing on architectural aspects such as structured task execution, information sharing mechanisms, human-in-the-loop integration, and failure management.

| Ref. | Structured Task Execution Mechanism | Information Sharing | HITL Management | Task-Level Failure Management |
|---|---|---|---|---|
| H-CoRE | Distributed executive system | - State messages<br>- TF transform | - High-level commands<br>- Online task monitoring | Online task-level/ motion-level replanning |
| [30] | Distributed Behavior Tree (BT) | - Robots' poses<br>- State messages | Start/Stop autonomous mission execution | - BT-based<br>- Task fault management not specified |
| [26] | Not considered (control architecture only) | Not specified | - Motion goals assignment<br>- Constraints injection | Not considered |
| [31] | Distributed state machine | - Sensor data<br>- Poses<br>- State messages | Not considered | Not considered |
| [16] | Distributed executive system | TF transforms | Not considered | Not specified |
| [32] | Not considered (centralized assignment of motion targets only) | Full robot state | Not specified | Not considered |
| [2] | Distributed state machine with dedicated cooperative planners | - Lightweight volumetric maps<br>- Target detections/positions | - High-level commands<br>- Online task monitoring | Not specified |
| [13] | - Central coordination<br>- Navigation tasks managed through BTs | Full robot state | High-level commands | - BT-based motion recovery and path-level replanning<br>- Task fault management not specified |
| [33] | Not considered (optimization-based assignment of atomic tasks only) | Application-dependent (peer-to-peer communication network) | Not considered | Not specified |

## 3. System Overview

The main components of the H-CoRE framework are illustrated in Figure 2. The human supervisor interacts with the Ground Control Station (GCS) by sending structured task commands to one or multiple agents and receiving feedback and status updates from the robotic system. Once a new goal is issued, the GCS dispatches tasks to the robots in the form of high-level symbolic commands, which are then instantiated, decomposed, and translated into motion primitives by the agents, depending on their current capabilities and operational state. Each agent's control software is organized into two primary layers: the *executive system* and the *motion stack*. The executive system receives high-level tasks from the GCS, allocates and schedules their execution, and monitors their progress. This module also communicates task execution status and feedback to the GCS, enabling the human operator to validate or cancel the execution of tasks. The motion stack, in turn, converts symbolic commands into motion commands for the robot's controller. Finally, in order to coordinate multi-robot missions, the agents rely on a shared knowledge based on ROS 2 Transformations Trees (`tf2`). All these components are further detailed in the following.

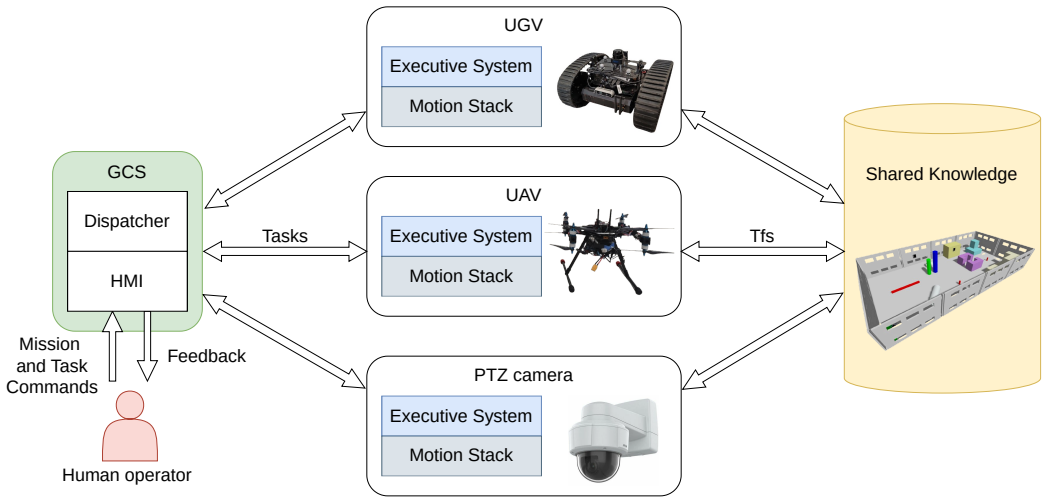



**Figure 2.** H-CoRE system overview. The GCS receives operator commands and provides feedback on task execution to the human. Multiple tasks are dispatched to the robots, which execute them autonomously using their motion stack. A shared knowledge, populated with relevant environmental features, supports coordination across the multi-robot team.

### *3.1. Ground Control Station and Human Interface*

In the H-CoRE framework, the GCS provides a human–robot interface that allows the operator to monitor task progress, receive real-time feedback from the agents, and retain supervisory control. The operator is continuously updated on task status and has the authority to initiate, modify, or terminate tasks as needed.

High-level commands from the operator are encoded in JSON format and are pre-assigned to one or more agents whose capabilities are compatible with the task requirements. For example, the task of finding an Aruco target (`id = 12`) in a specific zone of the map and within a deadline of 60 s can be represented as follows:

```
{
    "task_id": 12,
    "task_type": "find_target",
    "agents": ["uav","ugv"],
    "target_type": "aruco",
    "target_id": "123",
    "deadline": "60",
    "zone": "[(1, 1), (2, 1), (2,2), (1, 2)]"
}
```

Once received, the GCS dispatches this task to the indicated agents (UAV and UGV), converting it into the predicate:

```
find_target(aruco_target_12,((1,1),(2,1),(2,2),(1,2)),60,13)
```

which is transmitted across the ROS 2 network as a string message. Thus, the task specification, initially in a human-readable JSON format, is transformed into a more compact, structured string, which is then parsed and validated by the executive system of each robot. Notice that in the H-CoRE architecture, the GCS acts solely as a task dispatcher, with no complex centralized computation, making the rest of the system architecture fully decentralized.

### *3.2. Shared Knowledge*

In the proposed framework, all agents in the system have access to a shared knowledge, as depicted in Figure 2. Each agent continuously broadcasts its own pose, as well as the

poses of relevant features of the environment. To ensure global consistency and avoid conflicts, a dedicated data structure is used to collect and merge all detections into a unique pose (e.g., averaging values or keeping the latest only). These merged frames are persistently published as TFs by a specific node. This TF-based representation of the environment offers three key advantages. First, its sparsity significantly reduces the amount of data that must be exchanged between agents, especially when compared to dense representations such as octomaps or point clouds, and avoids the performance degradation observed when transmitting large data payloads over Wi-Fi [34,35]. Second, it inherently encodes preprocessed and semantically annotated environmental information, supporting interpretation and decision-making. Third, it provides a uniform and standardized way to share geometric data, natively supported in ROS 2, which enables agents to track the entire team's pose and efficiently execute structured, cooperative tasks. For example, a relevant feature might include a newly discovered target that requires inspection by another robot, which has a different viewpoint or sensing capabilities. The appearance of this feature in the transformation tree communicates the accomplishment of a task and can trigger the executive system of another robot, which will then schedule the new task based on its operational status.

### *3.3. Executive System*

Execution and supervision of complex activities are handled by assigning each team member an agent-specific executive system based on the cognitive architectures described in [16,36,37]. As illustrated in Figure 3, the executive system is composed of three modules: a Long Term Memory (LTM), a Working Memory (WM), and a Behavior-based System (BBS). The LTM is a Prolog knowledge base, storing the library of mission tasks and behaviors. These include complex, compound tasks such as the `cover(area)` or `inspect(target)`, which are defined prior to mission execution. These tasks are associated with symbolic constraints such as pre- and post-conditions, defining when the task is enabled or accomplished, respectively. Conversely, the WM maintains a dynamic, hierarchical representation of the *active* tasks during mission execution, from abstract descriptions to concrete, executable primitives (e.g., `flyto(waypoint)`). Finally, the BBS provides a symbolic interface to the low-level primitives available on each robotic platform (e.g., navigation, perception, and inspection modules). The BBS contains multiple behaviors, specific to each robotic agent, that are instantiated only if they are invoked in the WM as part of the current tasks' specifications. Once instantiated, each behavior continuously monitors its own state, checking if it is enabled or completed by tracking the task's pre-conditions and post-conditions, respectively. Notice that a behavior from the BBS represents a sensorimotor process that can be used to specify either an *action*, an *observation*, or a *communication*. Specifically, in this framework, actions are symbolic commands that are translated into physical movements by the motion stack. Observations monitor task-specific features of the environment, such as relations between poses or the appearance of significant frames in the map. Finally, communication processes send feedback data to the GCS, and in turn, to the human operator.

To deal with multiple concurrent tasks that cannot be executed simultaneously, the executive system uses three types of priorities: an operator-defined priority, a task-specific priority stored in the LTM, and a context-related priority. These priorities are used by a contention scheduling mechanism implemented in the BBS, as in [38]. Therefore, multiple behaviors continuously compete to acquire shared resources using a winner-takes-all principle. This approach fosters agents' autonomy by implementing a task scheduling mechanism that determines tasks' execution order and allows online adaptation to varying priorities and operative conditions. For instance, emergency missions, such as `emergency_land` or

`emergency_rtb` (return to base), are assigned the highest priority to ensure an immediate response to critical situations.

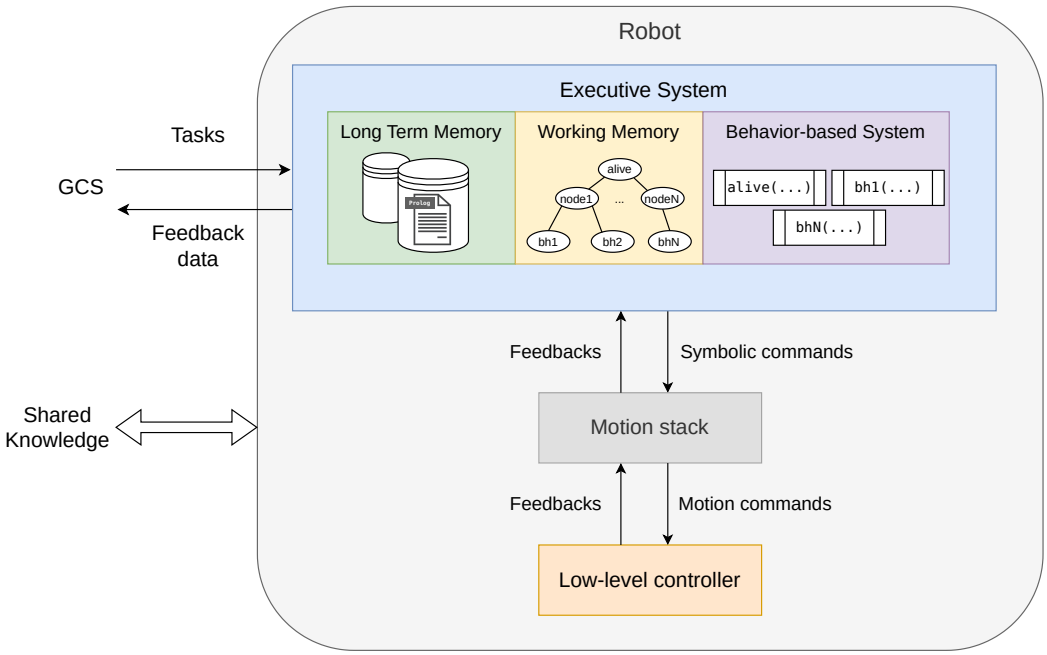



**Figure 3.** Representation of a robot's control architecture. In the executive system, symbolic tasks from the GCS are retrieved from the long-term memory, allocated in the working memory for actual execution through multiple behaviors. Symbolic commands are translated into motion primitives for the low-level controller.

To better detail this process, let us consider the `find_target` task for the aerial agent introduced in Section 3. This task is described by a *schema* contained in the LTM, which specifies the list parameters and sub-tasks as follows:

```
schema(find_target(T,(Z1,Z2,Z3,Z4),Deadline,ID),[
    [deadline(find_target(T,Deadline,ID),Deadline),1,["TRUE"]],
    [obs(T),1,["TRUE"]],
    [cover(Z1,Z2,Z3,Z4),1,[-T.exists]],
    [flyto(T.target,observe),2,[T.exists]],
    [photo(T,first,ID),2,[T.target.reached]],
    [trigger(T.second.ready,true),1,[T.first.confirmed]],
    [photo(T,second,ID),2,[T.second.ready]],
    [trigger(T.observed,true),1,[T.second.confirmed]] ],
    [T.observed] ).
```

In this case, the task parameters are a target `T` indicating the object of interest—typically a fiducial marker or feature identified through onboard vision; a list `(Z1, Z2, Z3, Z4)` specifying the four corners of the search area in the shared map frame; a `Deadline` as the maximum allowed time, in seconds, to complete the task; an `ID` corresponding to a unique identifier of the task. The sub-tasks and their associated pre-conditions are as follows:

- `deadline`: a task that removes `find_target` from the WM once the deadline has expired. This task is always enabled (`["TRUE"]`).
- `obs`: an observation behavior that monitors the presence of the target `T` in the shared knowledge. This behavior is always enabled (`["TRUE"]`).

- `cover`: an action behavior associated with an area coverage motion primitive in the motion stack. The behavior is enabled if the target `T` does not exist in the shared knowledge (`[-T.exists]`), meaning that it has not been found.
- `flyto`: an action behavior associated with a point-to-point motion in 3D space. The behavior is enabled if the target `T` exists in the shared knowledge (`[T.exists]`).
- `photo`: a communication behavior that sends to the GCS an image of the target `T` once found. The behavior is enabled if the target `T` is reached (`[T.target.reached]`).
- `trigger`: a sub-task that triggers temporal coordination between sequential actions. In this case, `T.observed` becomes true when the pre-condition `[T.first.confirmed]` or `[T.second.confirmed]` is satisfied, meaning that the human operator has confirmed the target presence in the photo.

Moreover, numerical values preceding the condition lists represent task priorities, while the post-condition `[T.observed]` signals task completion, and it is true when the whole sequence of sub-tasks is accomplished.

### *3.4. Motion Stack*

A robot's motion stack receives a symbolic command from the action behavior that has the highest priority and converts it into a motion primitive, sending commands to the low-level controller.

Each stack must (i) expose an interface to the executive system to parse and validate incoming high-level commands on the agent-specific topic, (ii) map these intents to the navigation and actuation stack via concise action/goal interfaces, (iii) implement a deterministic preemption mechanism to reset the ongoing task when a new command arrives, and (iv) publish real-time status and task symbolic feedback to support human task supervision.

Although all agents share this unified design philosophy and communication layer, each category of platform must implement a specialized motion stack, suited to its sensing configuration, mobility features, and mission-oriented tasks. Consequently, the motion stack is the only component of the architecture that requires robot-specific implementation. The corresponding motion layers for the robots examined in this work are presented in Section 4 in more detail.

## 4. Agent-Specific Motion Stacks

We specialize the H-CoRE system architecture for a heterogeneous team composed of UAV, UGV, and PTZ camera agents. Due to a strong heterogeneity—spanning aerial, ground, and fixed platforms—each robot requires a distinct motion layer.

Several components of these stacks were developed specifically for the H-CoRE framework to support coordinated multi-agent execution. In particular, the UAV motion stack includes a custom trajectory interpolation module, a novel dedicated interface with the PX4 autopilot, and a manager module designed to interact with the cognitive executive system. The PTZ camera stack integrates autonomous ROI detection, visual decoding, and closed-loop actuation, enabling the camera to operate as an active sensing agent within the team.

Whenever mature and well-established solutions were available, we deliberately relied on state-of-the-art software components rather than reimplementing existing libraries. For instance, the UAV path planner leverages `OMPL` for sampling-based planning and `FCL` for collision checking; while the UGV navigation stack uses the `Nav2` framework for global planning, local trajectory execution, and costmap management. This design allows the framework to combine newly developed modules with robust, community-tested components, while maintaining a uniform interface with the cognitive layer, across

all heterogeneous platforms. The remainder of this section describes the motion stacks implemented for each robot, with a focus on the integration of custom and standard components and on the motion primitives exposed to the higher-level executive systems.

### *4.1. UAV Motion Stack*

The UAV motion stack bridges the high-level task module with the low-level flight controller, leveraging the ROS 2 ecosystem and the PX4 autopilot; this design choice enables advanced autonomous behaviors (safe navigation in cluttered environments), while maintaining the reliability and safety guarantees of a commercial flight controller (flight modality switching, safety behaviors).

The motion stack is hierarchically structured into four stand-alone modules, as illustrated in Figure 4. The *Move Manager* (MM) module interprets high-level task commands and coordinates the system behavior. The *Path Planning* (PP) module provides global navigation through sampling-based search and map-based obstacle avoidance, while the *Trajectory Interpolation* (TI) module generates smooth, dynamically feasible motions with real-time reactive capability. Finally, the *PX4 Autopilot Bridge* manages coordinate transformations and protocol translation, serving as the interface between the planning modules and the flight controller.

The perception layer leverages the well-established RTAB-Map algorithm [39] to estimate the vehicle's state from onboard depth and visual data. Visual odometry is then fused with optical flow and inertial measurements within PX4's Extended Kalman Filter, closing the perception–control loop, as shown in Figure 4.

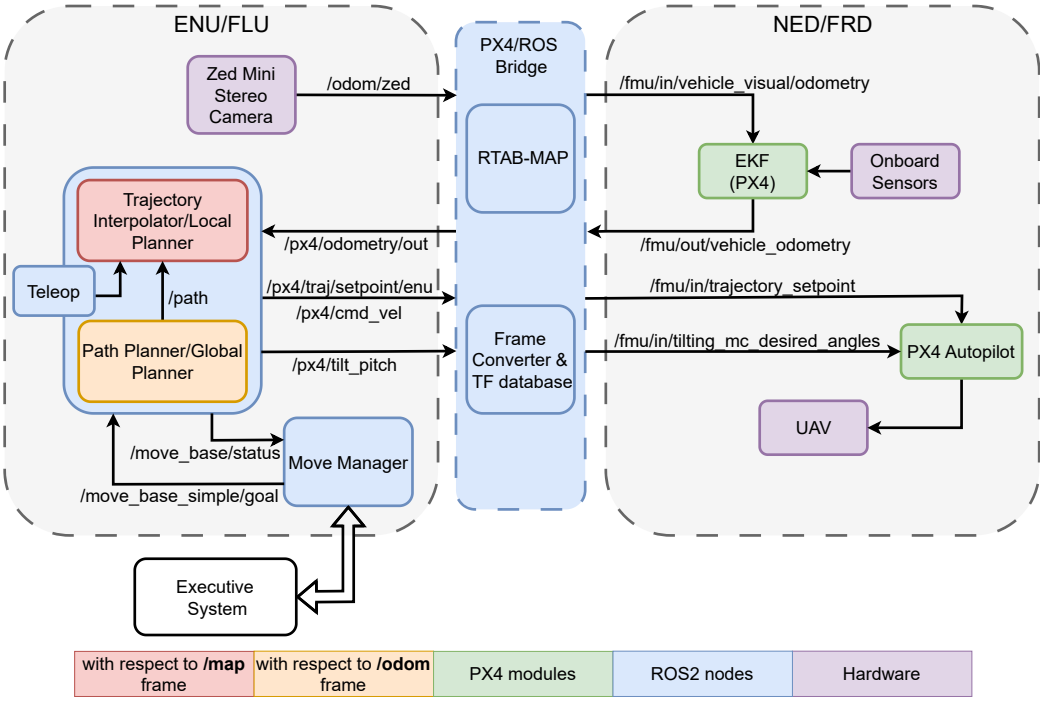



**Figure 4.** Complete UAV motion stack architecture, depicting the integration among perception, planning, and control, including frame conventions and main ROS 2 communication topics.

#### 4.1.1. Move Manager and Motion Primitives

The UAV *Move Manager* is the interface between the executive system and the navigation stack. The commands from the executive system are received via the ROS 2 topic `/seed/pdt/drone/command` and are parsed through a recursive descent mechanism that performs both syntactic and semantic verification. Support for nested and parametrized

structures enables highly expressive and flexible task specification. The commands implemented for the UAV correspond to the following motion primitives:

- `land` and `takeoff`: Vertical motions preserving the planar position, descending to the ground or ascending to a specified altitude, respectively. Both commands generate a single waypoint for the trajectory interpolation module.
- `flyto`: The basic navigation primitive consisting of a point-to-point navigation in 3D space. It forwards a Cartesian goal to the subsequent modules.
- `cover`: A boustrophedon coverage pattern with automatic vertex generation and sweep optimization, based on specified area dimensions [40]. This behavior is used to perform complete area coverage in the exploration phase.
- `loiter`: A circular path with configurable radius and angular resolution, particularly valuable for surveillance and inspection missions requiring sustained observation.
- `teleop`: Velocity control to allow direct human intervention.

The execution status of each primitive, as performed by subsequent modules in the motion stack, is continuously monitored by the Move Manager and forwarded to the executive system through the `/seed/pdt/drone/state` topic. This bidirectional communication enables human-in-the-loop operation and intervention capabilities, and multi-robot coordination, forming a critical component of our architecture.

#### 4.1.2. Path Planning Module

The global path planning module provides the base for autonomous navigation, and it is built on the Open Motion Planning Library (OMPL) [41] and the Flexible Collision Library (FCL) [42]. It generates a collision-free path by integrating obstacle data with sampling-based planning algorithms, primarily RRT*, within a bounded 3D state space (position and yaw). The module operates as an ROS 2 node that receives a navigation goal—typically a 3D pose with yaw angle—from the Move Manager via the `/move_base_simple/goal` topic. For the standard `flyto` primitive, the planner computes an obstacle-free sequence of waypoints from the current vehicle position to the target pose. The resulting path is then optimized to remove redundant waypoints and to have smooth transitions while maintaining safe clearance from the obstacles detected in the OctoMap representation. For specialized inspection primitives, the system concatenates additional geometric patterns to the base collision-free path: a circle of a specified radius in `loiter` mode and a boustrophedon (lawnmower) pattern [40] within the target polygonal area in `cover` mode. These patterns are then adjusted upon collision detection to ensure that the final motion remains collision-free. This implies that the resulting paths may deviate from ideal circular loiters or coverage patterns, as they are adapted to satisfy the constraints imposed by the environment. Finally, optimized waypoints are forwarded to the TI module for subsequent interpolation and execution.

In addition, the path planner module continuously monitors the current path—from the actual drone's position to the next waypoint—to verify whether changes in the Octomap, dynamically updated by the RTAB-Map node, result in any future collisions. If the path is no longer collision-free, a replanning process is triggered. All planning, success, and failure conditions are managed in this module through a finite state machine.

#### 4.1.3. Trajectory Interpolation Module

The UAV motion module integrates two complementary trajectory interpolation strategies. The *Base Trajectory Interpolator* generates position-, velocity-, and acceleration-level setpoints through reference filtering, enabling smooth waypoint interpolation with low computational overhead. This approach is primarily suited for nominal navigation in static environments or precision inspection tasks.

Conversely, the *Local Trajectory Interpolator* produces velocity commands using a potential-field formulation, enabling reactive responses to nearby obstacles and local disturbances without triggering frequent global replanning. While computationally slightly more demanding, this strategy is particularly effective during exploration or in dynamic environments. A concise comparison of the two approaches is reported in Table 3, while the detailed formulation and implementation aspects of both interpolation schemes are provided in Appendix A.

**Table 3.** Available trajectory interpolation methods' main features.

| | **Base Trajectory Interpolator** | **Local Trajectory Interpolator** |
|---|---|---|
| **Control Method** | Setpoints with reference filtering | Velocity cmds with potential fields |
| **Obstacle Response** | Stop and replan | Reactive |
| **Motion Characteristic** | Waypoints interpolation | Potential-based motion |
| **Computational Load** | Low (filter operations) | Medium (force computations) |
| **Best Application** | Static maps, precision inspection tasks | Dynamic environments, exploration |

#### 4.1.4. PX4 Autopilot Bridge

The PX4 Autopilot Bridge constitutes the final UAV motion stack component and serves as the integration layer between the high-level navigation modules and the low-level flight controller. Unlike standard MAVROS-based [43] solutions, the proposed bridge leverages the uXRCE-DDS [44] middleware to enable native ROS 2–PX4 communication with reduced latency and tighter synchronization.

The bridge ensures consistent bidirectional data exchange between ROS 2 and PX4 by handling full coordinate transformations between ENU/FLU (ROS conventions) and NED/FRD (PX4 conventions), including covariance rotation to preserve estimator consistency. Its functionality is organized into three concurrent processing pipelines: (i) visual odometry forwarding from the SLAM module to PX4, with proper frame alignment; (ii) state feedback conversion from the flight controller to the ROS ecosystem, including loop-closure correction propagation; and (iii) control command translation, converting trajectory or velocity setpoints into PX4-compatible offboard commands. In addition, pitch references are forwarded to a customized PX4 firmware for active sensor orientation.

Overall, the bridge provides a lightweight yet robust interface that guarantees frame consistency, estimator coherence, and seamless integration between the planning stack and the autopilot.

### *4.2. UGV Motion Stack*

The UGV agent provides persistent ground-level sensing and extended operational endurance. Unlike the UAV—whose motion stack is fully custom—the UGV navigation relies on the widely adopted `Nav2` framework [45], ensuring robust terrestrial navigation without platform-specific reimplementation. A detailed description of `Nav2` is beyond the scope of this work; only the configurations relevant to the experimental validation (Section 5) are summarized in Section 4.2.2. The UGV motion stack, illustrated in Figure 5, integrates *Nav2*, *RTAB-Map*, the *Move Manager*, and the *executive system*. In line with the H-CoRE design principles, community-maintained modules handle perception and navigation, while project-specific components ensure coordination within heterogeneous missions. Consistent with the UAV architecture, a dedicated Move Manager provides a unified interface for UGV motion primitives and task-level semantics.

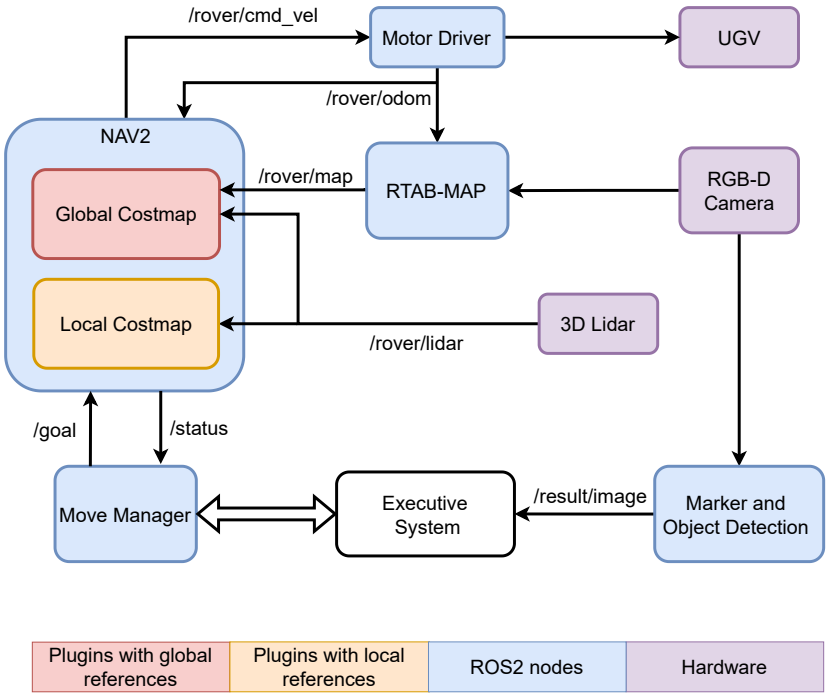



**Figure 5.** Complete UGV architecture, highlighting the interconnection among perception, mapping, and control, including the main ROS 2 communication topics.

#### 4.2.1. Move Manager and Motion Primitives

The UGV *Move Manager* is the motion-level interface for ground agents, implementing the same command grammar and execution semantics introduced for the UAV Move Manager, but specializing behaviors for this class of robot. Commands received on the ROS 2 topic `/seed/pdt/ugv/command` are parsed, validated, and mapped to platform-appropriate actions. The motion primitives implemented for the UGV are as follows:

- `goto`: A navigation primitive consisting of a point-to-point navigation on the ground plane, forwarding a validated 2D target pose to the navigation stack.
- `cover`: Exploration behavior consisting of generating and executing a boustrophedon sweep over a user-specified polygon, similarly to the UAV's primitive.
- `return`: Homing behavior driving the robot back to a designated station or fallback location, typically invoked at mission completion or resource thresholds.

As with the aerial counterpart, this module provides deterministic preemption, orderly cancellation, and lifecycle consistency when tasks change. Move intents are dispatched to the underlying `Nav2` pipeline. Execution status and health information are continuously reported on the topic `/seed/pdt/ugv/state` in the same way as the aerial system, enabling human supervision and multi-robot coordination.

#### 4.2.2. UGV Perception and Navigation Modules

Perception for the UGV is built upon RTAB-Map's incremental graph-based SLAM, configured for real-time onboard operation. Key settings include the following:

- *Height clamping:* obstacle points above 0.5 m are discarded to reduce memory footprint and update latency.
- *Point cloud preprocessing:* scan decimation and noise filtering (minimum cluster size: 20 points, radius: 0.2 m) mitigate spurious returns while preserving the geometric structure.
- *Grid parameters:* occupancy grid resolution of 0.05 m with range limits of 0.3–3.0 m, using normal-based segmentation to discriminate the traversable floor from obstacles.

The navigation architecture (Figure 5) relies on two complementary costmaps addressing distinct planning horizons. A rolling-window *local costmap* enables reactive obstacle avoidance using a 3D voxel layer derived directly from LiDAR data, while a globally referenced *static costmap* supports long-horizon planning via an occupancy grid layer sourced from RTAB-Map. Both costmaps share an inflation layer calibrated to the robot's dimensions and are configured with the parameters summarized in Table 4.

**Table 4.** Comparison between local and global costmap configurations in the UGV navigation stack.

| | **Local** | **Global** | | **Local** | **Global** |
|---|---|---|---|---|---|
| **General** | | | **Obstacle Detection** | | |
| Size | 6 × 6 m (rolling) | Map-bounded | Sensor | 3D LiDAR | 3D LiDAR |
| Resolution | 0.05 m | 0.05 m | Min range | 0.1 m | 0.1 m |
| Update frequency | 5 Hz | 2 Hz | Max range | 2.5 m | 2.5 m |
| Publish frequency | 2 Hz | 1 Hz | Raytrace range | 3.0 m | 3.0 m |
| Reference frame | `rover/odom` | `rover/map` | | | |
| **Layers** | | | **Inflation** | | |
| Static Layer (map) | — | ✓ | Robot radius | 0.22 m | 0.22 m |
| Obstacle Layer (LiDAR) | ✓(Voxel) | ✓ | Inflation radius | 0.2 m | 0.2 m |
| Inflation Layer | ✓ | ✓ | Cost scaling factor | 10.0 | 10.0 |

For trajectory execution, the system employs the Timed Elastic Band (TEB) local planner [46] configured for differential-drive kinematics. Velocity and acceleration limits are tuned conservatively to prevent wheel slip on indoor surfaces. The optimization objective prioritizes obstacle avoidance over strict trajectory smoothness, with a forward-motion bias to minimize backward maneuvers in cluttered environments.

The navigation pipeline operates across three coordinate frames: `rover/map` (SLAM output adjusted with loop closure), `rover/odom` (smooth, drift-prone odometry), and `rover/base_link` (robot body frame). Velocity smoothing is applied in the odometric frame to avoid control discontinuities during map corrections, ensuring stable motion even when loop closures introduce global pose adjustments.

### *4.3. PTZ Camera Motion Stack*

The PTZ camera serves as a static agent within the heterogeneous team, providing active vision capabilities for target acquisition, surveillance, and inspection. Its zoom capability, combined with high-precision orientation control, enables accurate target localization from a fixed spatial position. This motion stack has been implemented in different interconnected modules to allow communication with the executive system, control of the camera movements, and detection of notable targets. This design choice is aligned with the system's principles, since it allows reliance on community-maintained packages while ensuring proper communication and coordination with other agents.

Specifically, as illustrated in Figure 6, the PTZ motion stack architecture consists of four nodes: the *PTZ Manager Module*, the *Regions of Interest (ROI) Calculation Module*, the *Finder Module*, and the *Driver Module*.

The *PTZ Driver Module* is hardware-specific but provides a standardized software interface to the upper layers, abstracting low-level control and exposing consistent methods for velocity commands, absolute pitch–yaw–zoom goals, and camera parameter settings (e.g., iris, focus).

The *ROI Calculation Module* and the *Finder Module* process the camera's video stream to support exploration and inspection tasks. The former identifies sub-regions warranting

closer examination, while the latter performs object recognition and decoding, communicating the results back to the Move Manager for task-completion validation.

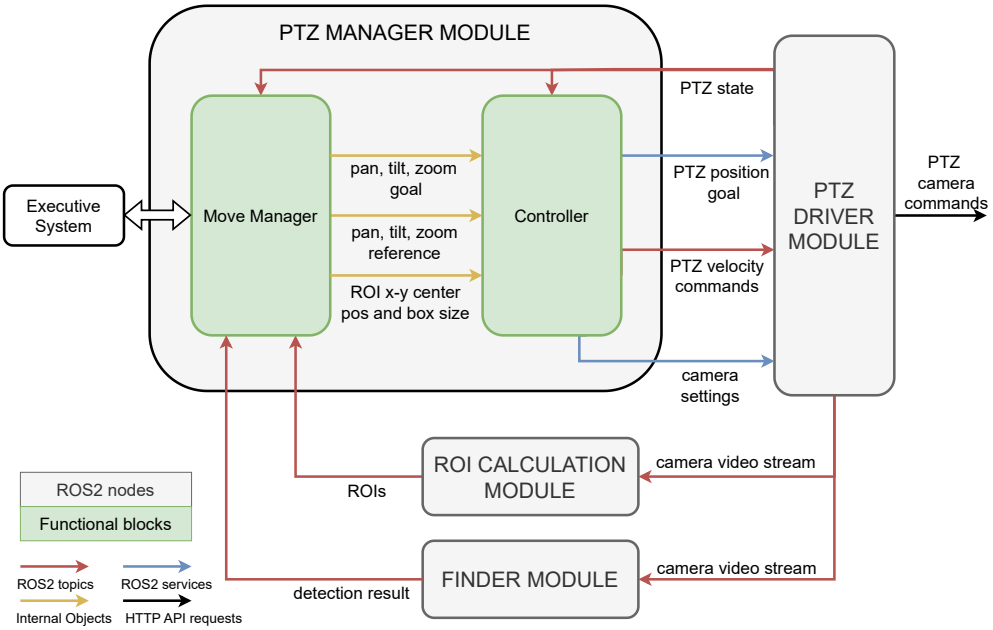



**Figure 6.** PTZ complete motion stack architecture. Note that the subdivision into functional blocks in the PTZ Manager Module is only conceptual; all the necessary functionalities are implemented in the corresponding FSM state action.

Finally, the *PTZ Manager Module* works as a central processing unit for the overall stack, implementing motion management and motion control functionalities. These are represented by the *Move Manager* and *Controller* functional blocks in Figure 6, which are effectively integrated within a single finite state machine (FSM).

#### 4.3.1. Move Manager and Motion Primitives

The Move Manager's operational logic is governed by a six-state FSM (see Figure 7). After initialization (`INIT`) and connection verification (`READY`), incoming commands trigger a transition to `DISPATCH` for interpretation, followed by the appropriate primitive state. The `manual` state, accessible only via direct topic commands, bypasses the executive system to allow human-operated absolute positioning. The FSM supports preemption of active states (`watchto`, `manual`, `cover`) to ensure responsiveness to dynamic mission commands (new commands trigger an immediate return to `DISPATCH`) and ensure compliance with the functional requirements of the H-CoRE framework.

The PTZ agent implements two motion primitives balancing area coverage with targeted inspection:

- `watchto`: Orients the camera toward a target frame.
- `cover`: Implements a hybrid search strategy for locating and identifying objects within polygonal areas.

In detail, the `watchto` primitive computes the homogeneous transformation from the camera's base frame to the specified target frame, retrieving and sending pan and tilt angles as absolute goals to the controller, while zoom is computed via the thin lens law based on estimated distance. Conversely, the `cover` algorithm dynamically switches between two operational modes: in the absence of a detected ROI, it operates in *Stochastic Exploration Mode*, generating random 3D target points within the area that are inspected using the error-based velocity controller; when an ROI is present, it transitions to *ROI-Driven Inspection*

*Mode*, selecting the highest-confidence detection, centering it using image-space error feedback, and adjusting zoom to maintain a target bounding box size. The action terminates either upon successful identification of the target or upon deadline expiration, publishing the results to the Ground Control Station.

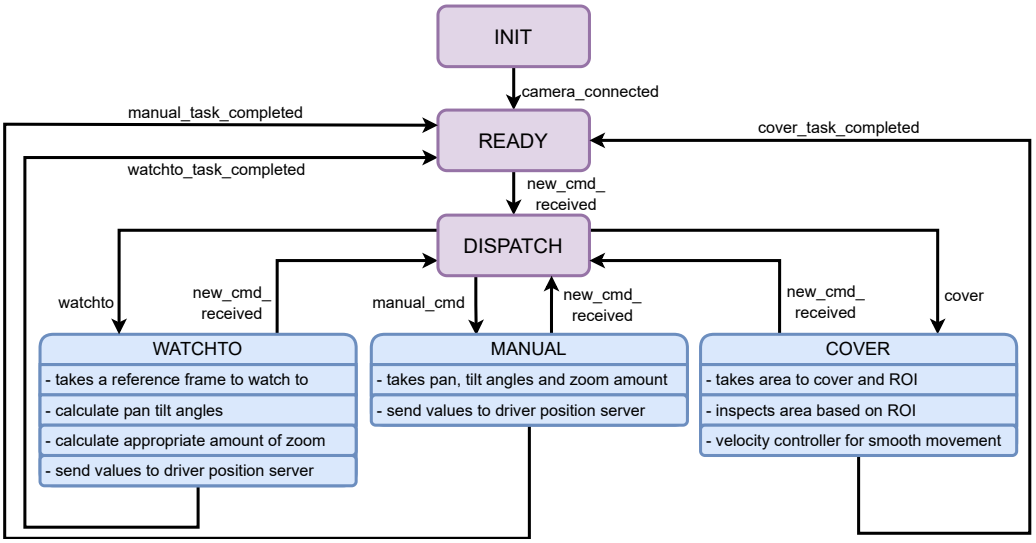



**Figure 7.** PTZ Move Manager finite state machine.

#### 4.3.2. PTZ-Based Perception and Control

The PTZ manager bridges high-level symbolic commands from the executive system with low-level camera control through two distinct modalities. In *position-based control*, absolute pan, tilt, and zoom goals are forwarded directly to the driver, which regulates camera joints without smoothing—suitable for precise pointing tasks such as the `watchto` primitive. In *velocity-based control*, two closed-loop controllers generate smooth trajectories for inspection behaviors like `cover`: a pan–tilt–zoom controller minimizes angular errors via proportional (P) action for pan and tilt and proportional-derivative (PD) action for zoom; an ROI-centering regulator instead operates in image space, minimizing the offset between the frame center and the selected ROI using proportional control, while adjusting zoom via PD action to maintain a target bounding box size that ensures adequate resolution for decoding.

Perception operates in parallel with motion control. ROI extraction is performed by a custom-trained YOLOv11 detector optimized for QR code recognition [47], trained on 1547 annotated images [48] and running on downsampled high-resolution frames. Detections exceeding a confidence threshold are published as normalized bounding boxes (XYWHN format). Concurrently, semantic validation and decoding rely on the ZBar library [49]; to minimize false positives, a detection is accepted only after the same payload is successfully decoded over multiple consecutive frames.

## 5. Experiments

This section presents the experimental results of the H-CoRE framework in a GNSS-denied environment. The validation is performed under the following assumptions: the system assumes that the environment boundaries are known, which aligns with typical exploration scenarios where area limits define the region of interest; the environment contains sufficient visual features to support reliable localization using visual–inertial SLAM; and a stable network connection over Wi-Fi is available to enable reliable broadcast communication. Two case studies are analyzed: the collaboration of a heterogeneous team—comprising a UAV, a UGV, and a PTZ camera—for structured task execution, and the

execution of tasks by a single drone using the custom motion stack described in Section 4.1. The evaluation assesses the system's ability to execute a complex mission autonomously while maintaining efficient coordination between heterogeneous agents and dynamic response to task changes.

A comprehensive video of the experiments is available at https://www.youtube.com/watch?v=32JMqU83qIw (accessed on 1 January 2026).

### 5.1. Experimental Setup

The heterogeneous robot team deployed for validation is composed of the following platforms.

- **UAV Agent**: An H-shaped coaxial tilting octacopter, equipped with a Zed Mini Stereo Camera (Stereolabs Inc., San Diego, CA, USA), and an ArkFlow optical flow sensor (ARK Electronics, Salt Lake City, UT USA) for localization and mapping. A Jetson Orin NX (NVIDIA Corporation, Santa Clara, CA, USA) functions as the companion computer.
- **UGV Agent**: Tracked ground platform equipped with a 3D Livox mid-360 LiDAR (Livox Technology Company Limited, Shenzhen, China) on top, and a frontal Realsense D435 RGB-D camera (Intel Corporation, Santa Clara, CA, USA) for terrestrial mapping and navigation. It features an onboard Intel NUC computer.
- **PTZ Camera Agent**: Axis M5525-E pan–tilt–zoom (Axis Communications AB, Lund, Sweden) IP camera with 10× optical zoom and 12× digital zoom, 1920 × 1080 pixels resolution, 30 fps, and adjustable iris aperture, brightness, and focus. Data are elaborated on a PC with Intel i7 CPU and Nvidia RTX 4060 GPU.

The experimental design closely follows the structure and requirements of the Leonardo Drone Contest [17] in terms of team composition, onboard sensors, target types (ArUco markers, QR codes, objects), and mission specifications. The arena dimensions were scaled down from the original competition environment due to laboratory space constraints while preserving the need for complete online mapping from an initially unknown state (only the arena dimensions are known). A faithful reproduction of the full-scale competition arena, including its original dimensions and obstacle layout, is available in the released simulation environment, as shown in Figure 8, allowing users to validate the framework under the original contest conditions.

To showcase the proposed framework capabilities, we deployed the heterogeneous team in an indoor arena, whose dimensions are 5 × 6 m with five fixed convex obstacles, as depicted in Figure 9. To simulate inspection targets, an object (teddy bear), five ArUco markers (IDs: 77, 69, 90, 6, 50), and one QR code (ID: 6) were placed in the scene. Because robots were required to actively search and identify a subset of these targets during the mission, both fiducial-marker detection [50] and YOLO-based object recognition [47] were used as visual detection algorithms. To demonstrate H-CoRE's collaboration capabilities, the UGV handled both object and ArUco-marker recognition, the UAV was limited to ArUco markers, and the PTZ camera was dedicated to QR codes.

The experiments consisted of multiple mission iterations performed with an identical setup and arena layout. All agents' operations started in an empty environmental map, necessitating complete online mapping without any prior knowledge, apart from the arena's outer dimensions. The arena was semantically partitioned into three disjoint zones, where each zone is defined as a polygon and identified by its four vertices $(Z1, Z2, Z3, Z4)$. These zones are used as attributes in the human-generated task specifications, as outlined in Section 3.1.

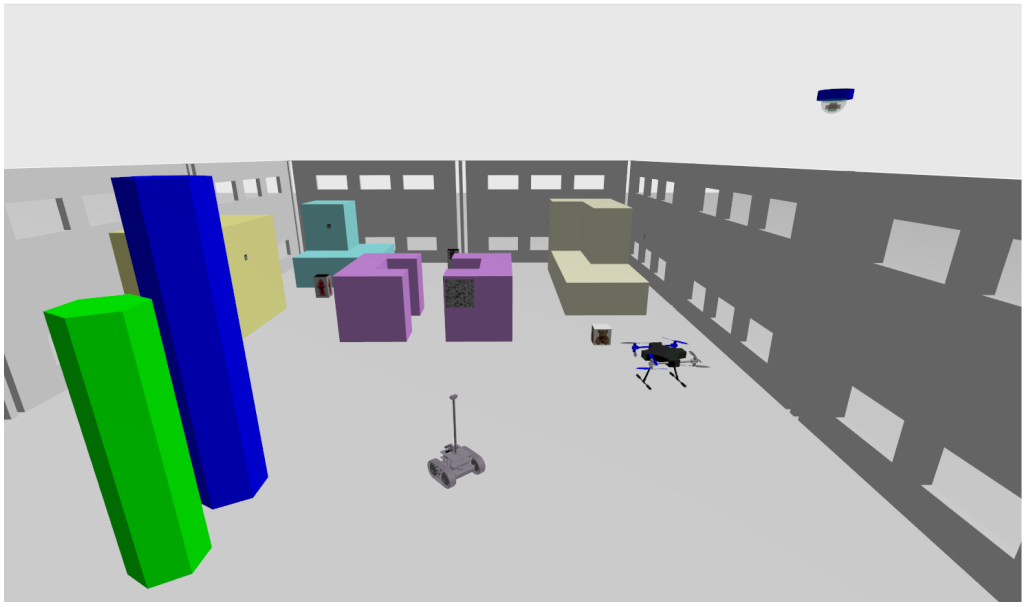


**Figure 8.** GZ simulation with the UAV, UGV, and PTZ agents deployed in a 10 × 20 scenario with obstacles and markers, mimicking the Leonardo Drone Contest arena [17].

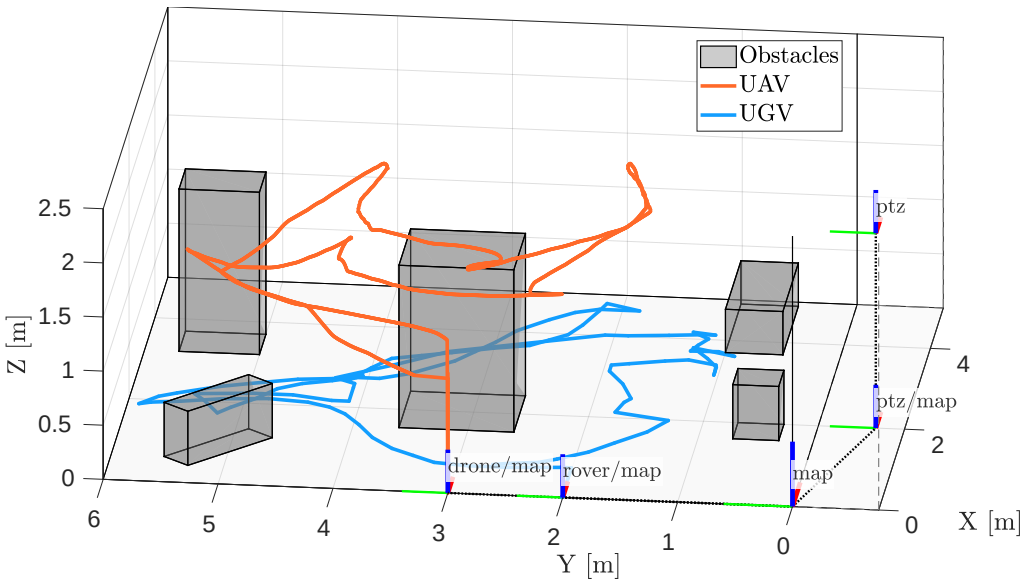



**Figure 9.** Experimental results: paths covered by the UAV (orange) and UGV (blue) agents. The depicted frames indicate the global map frame, the UAV's and UGV's starting poses, and the fixed PTZ position (placed outside the explorable environment).

Agents were then tasked to search for their assigned targets within the corresponding zone, operating either independently or in cooperation.

The system supports the *structured* task repertoire for mobile agents (UAV and UGV), summarized in Table 5. Each of these tasks is hierarchically decomposed into a sequence of sub-tasks, which can also be executed independently, with the full decompositions provided in Appendix B. In contrast, because no structured tasks were assigned to the PTZ, task decomposition was unnecessary, and the tasks issued by the GCS corresponded directly to the motion primitives.

**Table 5.** Structured task repertoire and agent assignment in the H-CoRE framework.

| Task Command | Description | Agent(s) |
|---|---|---|
| `emergency_land(Deadline, ID)` | Emergency landing routine with safety constraints | UAV |
| `emergency_rtb(Deadline, ID)` | Emergency return-to-base capability | UGV |
| `find_object(T, Deadline, ID)` | Object localization and continuous inspection | UAV, UGV |
| `find_target(T, (Z1, Z2, Z3, Z4), Deadline, ID)` | Systematic area coverage with target detection and multi-view inspection | UAV, UGV |
| `follow_sequence(S, Deadline, ID)` | Sequential waypoint navigation for structured exploration | UAV |

### *5.2. Case Study: Autonomous Mission Execution*

This case study examines the autonomous coordination capabilities of the H-CoRE framework through a representative multi-agent mission. A sample mission sequence, illustrated in Figure 10, shows the system's dynamic task scheduling and preemptive execution model. The GCS initially dispatches exploration tasks to the agents (`fly_to` and `find_target`) that start populating the shared knowledge database with visual detections. This proactive exploration is critical for mission efficiency; in fact, when the UAV receives the `find_target(69)` task, the marker has already been localized, and the drone can proceed directly to target inspection using the `fly_to` command, thereby circumventing the search phase implemented in the `cover` behavior (refer to Section 3.3 for the task's schema). After inspection, the drone remains idle, hovering on the spot, until a new task is dispatched.

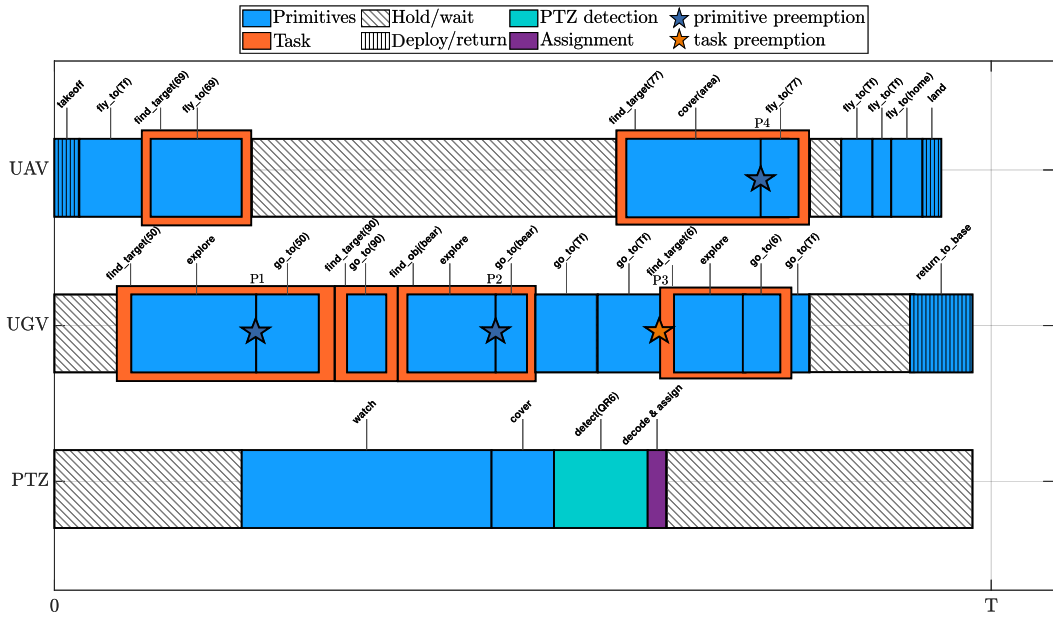



**Figure 10.** Mission timeline of the experiment, illustrating the sequence of assigned tasks and executed primitives for each agent. Structured tasks are shown as orange boxes, each containing the sequence of sub-tasks that were actually performed. Motion primitives are shown as black boxes and are marked with vertical lines when they correspond to deploy or return tasks. Hold and waiting phases are indicated with diagonal hatching, and preemption events $P1$, $P2$, $P3$, $P4$ are marked with stars.

A different behavior is observed in the UGV: when the command `find_target(50)` is issued without specifying a search zone, the `explore` behavior is triggered, consisting of a sequence of random `go_to` commands. This sequence is immediately preempted upon

detection of the marker and its registration in the shared memory. The task concludes once the rover reaches the target, completing an additional `go_to` sub-task.

Cooperative interaction emerges most clearly when the PTZ camera initiates the `find_target(6)` task, consisting of locating and inspecting the corresponding ArUco marker. In fact, when the stationary agent autonomously detects the QR code associated with marker 6 (violet box in Figure 10), the `find_target(6)` task is triggered and promptly assigned to the UGV. The UGV's executive system reacts in a preemptive manner, aborting the ongoing activity (`go_to(Tf)`) and allowing the ground agent to initialize the externally generated task. This sequence highlights the ability of the architecture to seamlessly integrate fixed and mobile sensing platforms into a unified and efficient inspection workflow.

It is noteworthy that the final two ArUco markers (IDs 77 and 50) were not detected during the initial exploration phase and were only localized during the related task initializations thanks to dedicated coverage patterns. The UAV successfully locates marker 77 during a systematic boustrophedon search of its assigned region, while the UGV discovers marker 50 while executing a similar pattern. These missions require more time compared to those involving pre-detected targets, clearly demonstrating the efficiency gained through the shared knowledge base and proactive exploration.

#### 5.2.1. Advanced Behavior Execution

The behavioral flexibility of the system was further demonstrated by performing an inspection task with the UAV. During the mission, the executive system triggers a high-priority `loiter` command following the detection of a teddy bear object by the UGV. The UAV immediately suspends the current task (a coverage one) and smoothly transitions to a loitering motion centered on the detected object, as depicted in Figure 11. Since the aerial platform features a tilting configuration and a single forward-facing camera, the loitering motion explicitly accounts for the platform's attitude: yaw and pitch references are computed to maintain the object within the camera's field of view while the UAV inclines toward the target. These references are computed from the teddy bear object pose, expressed in the world frame, and retrieved from the shared knowledge database.

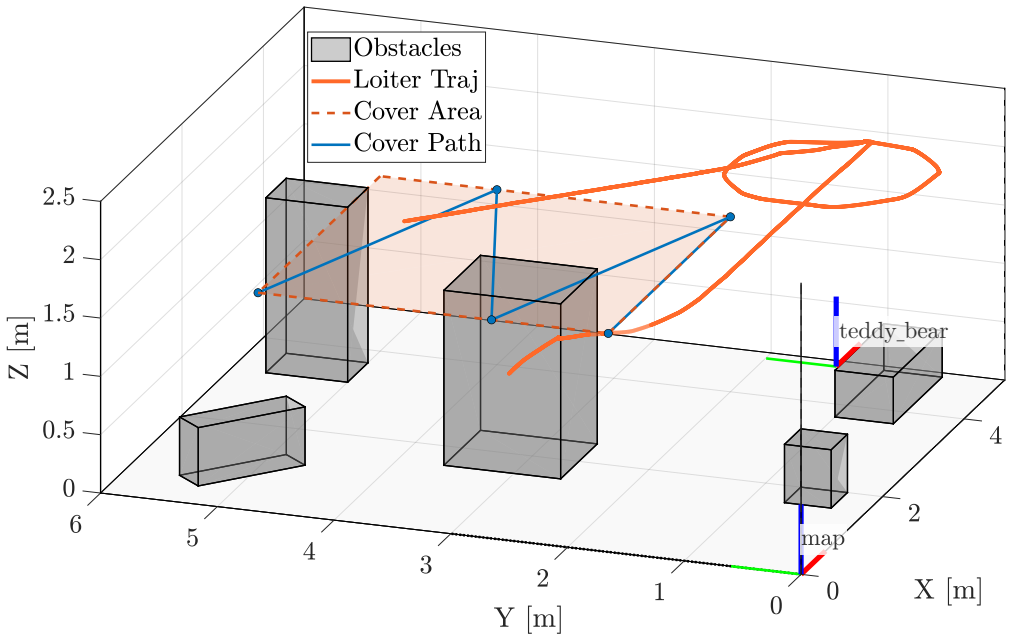



**Figure 11.** Adaptive behaviour execution: (blue) optimized boustrophedon coverage pattern with dynamic obstacle avoidance; (orange) loitering maneuver for sustained target observation using platform tilting capabilities.

This loitering movement, while available in all trials, was selectively dispatched based on mission priorities, occurring in approximately 60% of experimental runs. When executed, it successfully demonstrated the platform's ability to maintain persistent surveillance of static targets through coordinated position and attitude control.

#### 5.2.2. Results

The system's performance was evaluated across five experiments with randomized mission sequences. Across these trials, the system achieved a 93% success rate in target localization. The UGV demonstrated perfect performance (100% success rate), reliably locating all assigned targets. The UAV achieved an 87% success rate, with a single failure occurring during a `find_target(77)` mission where the marker's challenging orientation and the vehicle's forward-facing camera configuration prevented successful visual acquisition. Mission durations ranged from 12 to 15 min, remaining within the UAV battery life limits. The system maintained zero crash incidents across all trials, with robust collision avoidance ensuring safe operation in the cluttered experimental environment.

### *5.3. Lessons Learned and Discussion*

The implementation of the three motion stacks validated a core principle of the H-CoRE architecture: that heterogeneous coordination is best achieved through uniform execution and motion pipelines and interface standardization. Decomposing each stack into functionally distinct modules—manager, perception, path planning, control, and hardware abstraction—enabled us to accommodate platform-specific capabilities while preserving a uniform architectural skeleton. This modularity, combined with a shared command grammar and state reporting mechanism implemented in the Move Manager, allowed the executive system to interact with all agents through the same high-level primitives—`flyto`, `goto`, `watchto`, `cover`—and to monitor their status via consistent topics, regardless of whether the underlying implementation relied on Nav2, PX4, or custom controllers. By encapsulating domain-specific complexity behind standardized interfaces, we achieved multi-robot coordination despite radical differences in dynamics and sensing across aerial, ground, and fixed platforms. The integration of mature open-source solutions accelerated development and improved reliability, yet the project also revealed the limits of generic frameworks: custom modules, such as the tilting UAV control and the PTZ ROI-driven controller, proved essential where off-the-shelf components could not meet specific requirements.

Regarding the HITL management, the visual feedback collected during the operation (see Figure 12) proved essential not only for mission validation but also for preserving transparency in the interaction with the human supervisor. Detection results are currently made available in real time through dedicated topics, allowing low-latency access to perceptual information during execution. During the operations, the operator can still intervene through string-based commands interacting with the GCS to confirm target identification or request additional inspection actions. While this mechanism is sufficient for system monitoring, a more structured graphical interface would ease overall supervision, and it is therefore envisioned as part of future developments.

Another important aspect is the system's ability to be deployed in more challenging environments. In the experiments, we assumed that all agents can reliably communicate over the network; thus, additional development will be necessary to operate effectively in more complex scenarios, such as environments with intermittent communication. Moreover, as H-CoRE relies extensively on standard ROS 2 mechanisms, in particular, the TF tree for spatial reasoning and the DDS middleware for inter-agent communication, practical limits

on the number of agents do exist. However, these limits stem largely from the underlying components rather than from architectural constraints imposed by the framework itself.

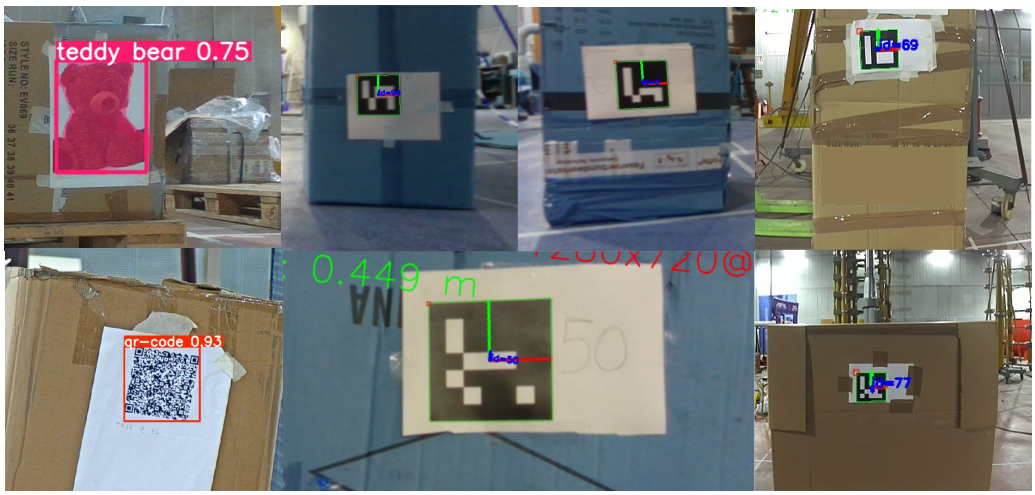



**Figure 12.** Visual verification samples: cropped images from successful target detections used for mission validation and human supervisor feedback.

On the other hand, the proposed framework was also deployed, with minimal adaptation, during the sixth edition of the Leonardo Drone Contest [17], held in a larger $20 \times 10$ m indoor arena with a different obstacle layout and different lighting and background conditions. The consistent performance across these environments, along with the team's third-place finish in the competition, provides external validation of the framework and supports its suitability for heterogeneous multi-robot operations in real-world scenarios.

Finally, the proposed framework may experience performance degradation due to the absence of centralized or distributed map fusion and global path planning. However, the overall architecture offers significant advantages in terms of flexibility, modularity, and the limited amount of standardized information that must be exchanged among agents, making it particularly suitable for scalable and heterogeneous multi-robot systems.

## 6. Conclusions

This paper presented H-CoRE, a heterogeneous multi-robot framework enabling autonomous cooperative exploration and visual inspection in complex and partially unknown environments. The architecture combines decentralized executive modules with a shared knowledge representation, allowing UAVs, UGVs, and PTZ sensors to coordinate seamlessly through standardized interfaces. The hierarchical task structure enables high-level tasks to be decomposed into platform-specific primitive actions, while still providing human supervisors with interpretable visual feedback and optional intervention capabilities.

Extensive experiments in both simulation and real-world deployments validated the robustness and flexibility of the proposed approach. The system executed fully autonomous missions lasting up to typical UAV endurance limits, dynamically adapting behaviors in response to environmental conditions, target detections, and mission priorities. The emphasis-based scheduling mechanism consistently resolved conflicts between exploration and inspection tasks, while the lightweight communication and memory-sharing paradigm ensured efficient coordination across the heterogeneous team. The successful deployment of H-CoRE in a large-scale competition environment further demonstrated its operational readiness.

Thanks to its modular design and primitive-based action abstraction, the framework is readily extensible to broader use cases, including industrial inspection and infrastructure monitoring. Future work will focus on expanding the set of supported agent types—such

as legged robots for improved mobility or morphing robots [51–53]—and on integrating additional inspection modalities, including contact-based assessment tools. In addition, although all system data can already be visualized using the basic monitoring and visualization tools provided by ROS 2, managing a system of this complexity through such interfaces remains challenging. For this reason, future work will investigate the design and evaluation of a dedicated graphical user interface tailored to the framework's requirements, enabling more effective high-level human supervision. Such an interface would not only allow interaction with the agents at an abstract level but also support visualization of task execution states, as well as confirmation requests and task repetition when needed. To conclude, further research will also explore multi-modal perception, richer semantic mapping, and AI-driven anomaly detection to increase autonomy, resilience, and situational awareness in complex missions.

**Author Contributions:** Software, S.D., F.P., V.S., A.D.C., S.C., P.M., and R.C.; methodology, investigation, and validation R.C., S.D., V.S., F.P., S.C., P.M., and A.D.C.; data curation, S.D.; formal analysis and writing, S.D., F.P., P.M., S.C., and R.C.; review and editing, F.P., V.L., R.C., and A.F.; data visualization, S.D.; supervision, V.L. and A.F.; project administration, V.L.; resources and funding acquisition, R.C. and V.L. All authors have read and agreed to the published version of the manuscript.

**Funding:** This work was supported by the EU Horizon INVERSE (grant 101136067), by the INVITALIA NEMESI - CUP C67G22000420008, and by SPACE IT UP (ASI MUR Contract 2024-5-E.0 - CUP I53D24000060005) projects.

**Institutional Review Board Statement:** Not applicable.

**Informed Consent Statement:** Not applicable.

**Data Availability Statement:** Data will be made available on request.

**Acknowledgments:** The authors gratefully acknowledge Leonardo S.p.A. for inspiring the heterogeneous team configuration and the case studies. The research presented herein was conducted while Vincenzo Scognamiglio was enrolled at PRISMA Lab. KRUSH Labs B.V. was not involved in the development of this article or its content.

**Conflicts of Interest:** Author Vincenzo Scognamiglio is employed by the company KRUSH Labs B.V., while Author Alessandro De Crescenzo is employed by the company Neabotics. The remaining authors declare that the research is conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

## Appendix A. Solutions for UAV Trajectory Generation

As depicted in Section 4.1, the proposed UAV motion stack offers two different trajectory interpolation strategies. In the following, we analyze them in detail.

### *Appendix A.1. BaseTrajectory Interpolator*

The Base Trajectory Interpolator converts a discrete waypoint sequence into a smooth, dynamically feasible trajectory. This module functions as a computationally lightweight *motion filter*: it implements a second-order closed-loop system that generates an ideal acceleration reference, then processes it through a hierarchical constraint chain (jerk, acceleration, and velocity limits) to ensure motion feasibility.

For each translational axis $i \in \{x, y, z\}$, the desired acceleration is computed via a PD control law acting on the tracking error $e_i(t) = p_i^{fb}(t) - p_i^{ref}(t)$:

$$a_i^{des}(t) = \omega_i^2(t) e_i(t) - 2\zeta_i \omega_i v_i^{ref}(t), \tag{A1}$$

where $\omega_i$ and $\zeta_i$ denote the natural frequency and damping ratio, $p_i^{fb}$ is the $i$-th component of the UAV's actual position feedback, while $p_i^{ref}$ is the current reference position. To enforce dynamic feasibility, the desired acceleration is filtered through a jerk-, acceleration-, and velocity-limited integrator cascade. This conditioned integration scheme, commonly adopted in jerk-limited trajectory generation [54–57], ensures that all generated references comply with the platform's dynamic constraints. For multiple waypoints, an anticipatory switching mechanism ensures continuous motion without intermediate stops. The complete approach is summarized in Algorithm A1 and on the Github (https://github.com/Prisma-Drone-Team/uav_motion_stack.git, accessed on 1 January 2026) page of the UAV motion stack.

Yaw is interpolated under the same dynamic constraints and aligned with the direction of motion during exploration, transitioning smoothly to the final task orientation near the goal. This strategy enhances perceptual robustness in vision-based navigation.

**Algorithm A1** Base Trajectory Interpolator Algorithm

**Input:** UAV position $\mathbf{p}$, desired goal position $\mathbf{p}^{cmd}$; parameters $(\omega_i, \zeta_i, a_i^{max}, v_i^{max}, j_i^{max})$, $\forall i = \{x, y, z\}$; timestep $\Delta t$; initialized with $p_i^{ref}(0) = p_i^{cmd}$, $v_i^{ref}(0) = 0$ and $a_i^{ref}(0) = 0$.
**for** $i = \{x, y, z\}$ **do**
  Desired acceleration: $a_i^{des}(t) = \omega_i^2(p_i^{fb}(t) - p_i^{ref}(t)) - 2\zeta_i\omega_i v_i^{ref}(t)$
  Jerk: $j_i = (a_i^{des} - a_i^{ref})/\Delta t$
  Limit jerk:
  **if** $|j_i| > j_i^{max}$ **then**
    $j_i = \text{sign}(j) \cdot j_i^{max}$
  **end if**
  $a_i^{des} = a_i^{ref} + j_i \cdot \Delta t$
  Enforce acceleration limit:
  **if** $|a_i^{des}| > a_i^{max}$ **then**
    $a_i^{ref} = \text{sign}(a_i^{des}) \cdot a_i^{max}$
  **else**
    $a_i^{ref} = a_i^{des}$
  **end if**
  Integrate to compute velocity: $v_i^{des} = v_i^{ref} + a_i^{ref} \cdot \Delta t$
  Enforce velocity limit:
  **if** $|v_i^{des}| > v^{max}$ **then**
    $v_i^{ref} = \text{sign}(v_i^{des}) \cdot v_i^{max}$
  **else**
    $v_i^{ref} = v_{des}$
  **end if**
  Integrate to compute position: $p_i^{ref} = p_i^{ref} + v_i^{ref} \cdot \Delta t$
**end for**

Finally, an additional pitch reference is generated for tilting platforms. This capability is particularly useful in inspection tasks, allowing the onboard camera to be oriented toward a point of interest by automatically calculating the necessary pitch from the target's relative position.

*Appendix A.2. Local Trajectory Interpolator*

The Local Trajectory Interpolator implements real-time reactive obstacle avoidance using Artificial Potential Fields (APFs) [58]. It operates as a local refinement layer between successive waypoints of the collision-free global path generated by the RRT* planner. The UAV motion is governed by an artificial force obtained as the sum of an attractive field that drives the vehicle along the global path and two repulsive fields that push it away

from nearby obstacles. In particular, the attractive component steers the UAV toward a lookahead point $p^g$, corresponding to the closest waypoint on the global trajectory. This force is defined as

$$f_{\mathrm{a}} = k_{\mathrm{a}} \tanh\left(\frac{\|p^g - p^{fb}\|}{d_{\max}}\right) \frac{(p^g - p^{fb})}{\|p^g - p^{fb}\|}, \tag{A2}$$

where $k_{\mathrm{a}}$ is a proper gain, and $d_{\max}$ is a distance corresponding to maximum scaling.

Repulsive force fields are computed from Octomap data and the current RGB-D point cloud, treating voxels and points as obstacles. In both cases, the exerted repulsive force is expressed as

$$f_{\mathrm{r},i} = \begin{cases} k_{\mathrm{r}} \left(\frac{r_{\mathrm{inf}} - d_i}{r_{\mathrm{inf}}}\right)^2 \frac{1}{d_i^2} \hat{u}_i, & d_i < r_{\mathrm{inf}} \\ 0 & d_i \geq r_{\mathrm{inf}} \end{cases} \tag{A3}$$

where $k_{\mathrm{r}}$ is a positive gain, $r_{\mathrm{inf}}$ is the maximum influence radius beyond which the force is set to zero, $d_i$ is the distance between the UAV and the $i$-th obstacle, and $\hat{u}_i$ is the associated unit vector. Different parameters are adopted for map-based and point-cloud-based terms to balance global structural awareness and short-range reactivity.

The total force, computed by summing all contributions, is interpreted as a desired translational velocity, following a standard APF-based navigation approach [58] (Sec. 12.6). Specifically, the velocity command is scaled based on the yaw error and computed as

$$v_{\mathrm{cmd}} = e^{-k_\psi |e_\psi|} f^o, \tag{A4}$$

with $k_\psi > 0$ and $e_\psi$ being the difference between the current and the desired yaw. This modulation reduces translational velocity when the vehicle is significantly misaligned with the direction of motion, facilitating reorientation before progressing along the path. Such behavior further supports effective localization, mapping, and obstacle avoidance in unknown environments, even when relying on a single forward-facing camera sensor.

## Appendix B. Task Primitive Decompositions

This appendix details the primitive decompositions of each structured task implemented in the released H-CoRE framework. Each task is described by a Prolog schema, contained in the agents' LTM, which translates high-level commands into executable sequences of primitive actions, which depend on the specific robotic platform capabilities.

### *Appendix B.1. UAV Task Primitives*

`find_object(T, Deadline, ID)`

The UAV performs a complete circular trajectory on an object initially detected by the UGV. The task begins with a deadline timer that ensures automatic termination if the task exceeds its allocated time. Once the object observation is activated, the UAV enters a circular loitering pattern around the target's estimated position, maintaining continuous visual coverage. During this loitering phase, the UAV captures a continuous stream of imagery to support detailed inspection.

**Primitive sequence:**

1. `deadline(find_object(...), Deadline)`: Task timer initialization with automatic timeout
2. `obs(T)`: Object observation and tracking activation
3. `flyto(circle(T))`: Circular loitering execution (conditional on: T exists, UAV flying, no failures)
4. `photo(T, continuous, ID)`: Continuous visual documentation during loitering

**Termination condition:** `circle(T).done`

`find_target(T, (Z1,Z2,Z3,Z4), Deadline, ID)`

This task implements a two-phase search-and-inspect strategy. The UAV first systematically covers a defined operational zone using a bustrofedonic pattern while monitoring for target detection. Upon target identification, the UAV transitions to an inspection phase, navigating to an optimal observation point. The inspection protocol captures two complementary images with a 2 s interval, allowing for multi-view analysis and potential 3D reconstruction. A final confirmation delay ensures observation validation before task completion.

**Primitive sequence:**

1. `deadline(find_target(...), Deadline)`: Task timing constraint
2. `obs(T)`, `obs(T.target)`: Dual monitoring of target and optimal observation point
3. `cover(Z1,Z2,Z3,Z4)`: Systematic area coverage (executed when target not detected)
4. `flyto(T.target, observe)`: Precision navigation to inspection point (when target detected)
5. `photo(T, first, ID)`: Initial inspection capture
6. `timer(T.second.ready, true, 2.1)`: Controlled interval for second observation
7. `photo(T, second, ID)`: Complementary inspection angle
8. `timer(T.observed, true, 0.1)`: Final observation validation

**Termination condition**: `T.observed`

`follow_sequence(S, Deadline, ID)`

This task executes a sequential navigation of waypoints, preserving the visit order. It is designed for structured inspection, as each waypoint corresponds to a previously detected marker.

**Primitive sequence:**

1. `deadline(follow_sequence(...), Deadline)`: Overall task timing
2. `hardSequence(S, fs)`: Sequential execution of waypoint list where each waypoint includes:
   - `obs(T)`: Waypoint observation initialization
   - `flyto(T.target, observe)`: Navigation to waypoint
   - `photo(T, once, ID)`: Waypoint documentation
   - `timer(T.observed, true, 0.1)`: Navigation confirmation

**Termination condition**: `hardSequence(fs).done`

*Appendix B.2. UGV Task Primitives*

`emergency_rtb(Deadline, ID)`

This safety-critical task implements an immediate return to a designated safe location when emergency conditions are detected. The task employs the highest priority level (emphasis = 20) to override any concurrent tasks, ensuring rapid response to critical situations. The navigation directly targets the pre-defined base location without intermediate behaviors.

**Primitive sequence:**

1. `deadline(emergency_rtb(...), Deadline)`: Emergency response timing
2. `goto(rover/map)`: High-priority (emphasis = 20) direct navigation to base

**Termination condition**: `rover/map.reached`

`find_object(T, Deadline, ID)`

The UGV performs ground-level object search with adaptive behavior based on detection status. While the object remains undetected, the UGV executes exploratory navigation to systematically search the operational area. Upon object detection, the platform transitions to documentation mode, capturing a single high-resolution image for target verification.

**Primitive sequence:**

1. `deadline(find_object(...), Deadline)`: Search operation timing
2. `obs(T)`: Object detection system activation
3. exploration strategy until the object is detected:
   - 3.1. `cover(Z1,Z2,Z3,Z4)`: Terrestrial area coverage
   - 3.2. `explore`: Systematic area exploration with `go_to` sub-tasks (when `zone = null`)
4. `photo(T, once, ID)`: Single-shot documentation (when object detected)

**Termination condition**: `T.once.confirmed`

`find_target(T, (Z1,Z2,Z3,Z4), Deadline, ID)`

This task mirrors the UAV's search-and-inspect strategy but adapts it for ground operations.

**Primitive sequence:**

1. `deadline(find_target(...), Deadline)`: Task timing constraint
2. `obs(T)`, `obs(T.target)`: Target and ground observation point monitoring
3. exploration strategy until target is detected:
   - 3.1. `cover(Z1,Z2,Z3,Z4)`: Terrestrial area coverage
   - 3.2. `explore`: Systematic area exploration with `go_to` sub-tasks (when `zone = null`)
4. `goto(T.target, observe)`: Ground navigation to inspection point (when target detected)
5. `photo(T, first, ID)`: Initial ground-level inspection
6. `timer(T.second.ready, true, 2.1)`: Interval for complementary angle
7. `photo(T, second, ID)`: Secondary inspection from ground perspective
8. `timer(T.observed, true, 0.1)`: Observation validation

**Termination condition**: `T.observed`

## References


1. Sales, A.; Mira, P.; Maria Nascimento, A.; Brandão, A.; Saska, M.; Nascimento, T. Heterogeneous Multi-Robot Systems Approach for Warehouse Inventory Management. In *Proceedings of the 2023 International Conference on Unmanned Aircraft Systems (ICUAS)*; IEEE: Piscataway, NJ, USA, 2023; pp. 389–394.
2. Petrlík, M.; Petráček, P.; Krátký, V.; Musil, T.; Stasinchuk, Y.; Vrba, M.; Báča, T.; Heřt, D.; Pecka, M.; Svoboda, T.; et al. UAVs Beneath the Surface: Cooperative Autonomy for Subterranean Search and Rescue in DARPA SubT. *IEEE Trans. Field Robot.* **2025**, *2*, 643–689. [CrossRef]
3. Yucesoy, E.; Balcik, B.; Coban, E. The role of drones in disaster response: A literature review of operations research applications. *Int. Trans. Oper. Res.* **2025**, *32*, 545–589.
4. Chang, Y.; Cheng, Y.; Manzoor, U.; Murray, J. A review of UAV autonomous navigation in GPS-denied environments. *Robot. Auton. Syst.* **2023**, *170*, 104533. [CrossRef]
5. Akhihiero, D.; Olawoye, U.; Das, S.; Gross, J.N. Cooperative Localization for GNSS-Denied Subterranean Navigation: A UAV–UGV Team Approach. *Navig. J. Inst. Navig.* **2024**, *71*, navi.677. [CrossRef]
6. Duan, S.; Shi, Q.; Wu, J. Multimodal Sensors and Machine-Learning-Based Data Fusion for Advanced Robots. *Adv. Intell. Syst.* **2022**, *4*, 2200213. [CrossRef]
7. Verma, J.K.; Ranga, V. Multi-Robot Coordination: Analysis, Taxonomy, Challenges and Future Scope. *J. Intell. Robot. Syst.* **2021**, *102*, 10. [CrossRef]
8. Munasinghe, I.; Perera, A.; Deo, R.C. A Comprehensive Review of UAV-UGV Collaboration: Advancements and Challenges. *J. Sens. Actuator Netw.* **2024**, *13*, 81. [CrossRef]

9. Iñigo-Blasco, P.; Diaz-del Rio, F.; Romero-Ternero, M.C.; Cagigas-Muñiz, D.; Vicente-Diaz, S. Robotics software frameworks for multi-agent robotic systems development. *Robot. Auton. Syst.* **2012**, *60*, 803–821. [CrossRef]
10. Chovet, L.P.; Garcia, G.M.; Bera, A.; Richard, A.; Yoshida, K.; Olivares-Mendez, M.A. Performance Comparison of ROS 2 Middlewares for Multi-robot Mesh Networks in Planetary Exploration. *J. Intell. Robot. Syst.* **2025**, *111*, 18.
11. Gil-Castilla, M.; Poma, A.R.; Caballero, A.; Ollero, A. A Cooperative Multi-Uav Framework for Bridge Inspection. In *Proceedings of the 2025 International Conference on Unmanned Aircraft Systems (ICUAS)*; IEEE: Piscataway, NJ, USA, 2025; pp. 408–415.
12. Araújo, A.G.; Pizzino, C.A.P.; Couceiro, M.S.; Rocha, R.P. A Multi-Drone System Proof of Concept for Forestry Applications. *Drones* **2025**, *9*, 80. [CrossRef]
13. Yumbla, F.; Fajardo-Pruna, M.; Piguave, A.; Ronquillo, D.; Ortiz, R.; Choi, J.B.; Díaz, G.; Pañeda, X.G.; Moon, H. An Open-Source Multi-Robot Framework System for Collaborative Environments Based on ROS2. *IEEE Access* **2025**, *13*, 16288–16302.
14. Denguir, M.; Touir, A.; Gazdar, A.; Qasem, S. Toward a Generic Framework for Mission Planning and Execution with a Heterogeneous Multi-Robot System. *Sensors* **2024**, *24*, 6881. [CrossRef] [PubMed]
15. Boiteau, S.; Vanegas, F.; Gonzalez, F. Framework for Autonomous UAV Navigation and Target Detection in Global-Navigation-Satellite-System-Denied and Visually Degraded Environments. *Remote Sens.* **2024**, *16*, 471.
16. Scognamiglio, V.; Caccavale, R.; Finzi, A.; Lippiello, V. A Scalable Multi-robot System for Cooperative Exploration. In *Proceedings of the European Robotics Forum 2024*; Secchi, C., Marconi, L., Eds.; Springer: Cham, Switerland, 2024; pp. 200–204.
17. Leonardo S.p.A. Leonardo Drone Contest. 2025. Available online: https://www.leonardo.com/it/innovation-technology/open-innovation/drone-contest (accessed on 15 December 2025).
18. Roggi, G.; Meraglia, S.; Lovera, M. Leonardo drone contest autonomous drone competition: Overview, results, and lessons learned from politecnico di milano team. *J. Intell. Robot. Syst.* **2023**, *108*, 23. [CrossRef]
19. Godio, S.; Marino, F.; Minervini, A.; Primatesta, S.; Chiaberge, M.; Guglieri, G. Autonomous drones in GNSS-denied environments: Results from the Leonardo Drone Contest. In *Proceedings of the 2023 IEEE 10th International Workshop on Metrology for AeroSpace (MetroAeroSpace)*; IEEE: Piscataway, NJ, USA, 2023; pp. 1–6.
20. Roggi, G.; Meraglia, S.; Lovera, M. Leonardo drone contest 2021: Politecnico di milano team architecture. In *Proceedings of the 2022 International Conference on Unmanned Aircraft Systems (ICUAS)*; IEEE: Piscataway, NJ, USA, 2022; pp. 676–685.
21. Mugnai, M.; Teppati Losé, M.; Herrera-Alarcón, E.P.; Baris, G.; Satler, M.; Avizzano, C.A. An efficient framework for autonomous UAV missions in partially-unknown GNSS-denied environments. *Drones* **2023**, *7*, 471.
22. Bianchi, L.; Carnevale, D.; Del Frate, F.; Masocco, R.; Mattogno, S.; Romanelli, F.; Tenaglia, A. A novel distributed architecture for unmanned aircraft systems based on Robot Operating System 2. *IET Cyber-Syst. Robot.* **2023**, *5*, e12083.
23. Van Manen, B.R.; Ter Maat, G.; Boe, M.; Mersha, A.Y. The BEAST: Modular Open-Source Framework for BVLOS Drone Flights with Long-Term Autonomy. In *Proceedings of the 2025 International Conference on Unmanned Aircraft Systems (ICUAS)*; IEEE: Piscataway, NJ, USA, 2025; pp. 457–464.
24. Murillo, J.; Montes, M.; Zahinos, R.; Trujillo, M.; Viguria, A.; Heredia, G. Simplifying Autonomous Aerial Operations: LUCAS, a Lightweight Framework for UAV Control and Supervision. In *Proceedings of the 2025 International Conference on Unmanned Aircraft Systems (ICUAS)*; IEEE: Piscataway, NJ, USA, 2025, pp. 854–861.
25. Scognamiglio, V.; Caccavale, R.; Merone, P.; Crescenzo, A.D.; Ruggiero, F.; Lippiello, V. Autonomous Visual Inspection of Industrial Plants Using Unmanned Aerial Vehicles. In *Proceedings of the 2024 International Conference on Unmanned Aircraft Systems (ICUAS), Chania-Crete, Greece*; IEEE: Piscataway, NJ, USA, 2024; pp. 1148–1154.
26. Baca, T.; Petrlik, M.; Vrba, M.; Spurny, V.; Penicka, R.; Hert, D.; Saska, M. The MRS UAV System: Pushing the Frontiers of Reproducible Research, Real-world Deployment, and Education with Autonomous Unmanned Aerial Vehicles. *J. Intell. Robot. Syst.* **2021**, *102*, 26. [CrossRef]
27. Kottege, N.; Williams, J.; Tidd, B.; Talbot, F.; Steindl, R.; Cox, M.; Frousheger, D.; Hines, T.; Pitt, A.; Tam, B.; et al. Heterogeneous Robot Teams with Unified Perception and Autonomy: How Team CSIRO Data61 Tied for the Top Score at the DARPA Subterranean Challenge. *Field Robot.* **2024**, *4*, 313–359. [CrossRef]
28. Rouček, T.; Pecka, M.; Čížek, P.; Petříček, T.; Bayer, J.; Šalanský, V.; Heřt, D.; Petrlík, M.; Báča, T.; Spurný, V.; et al. DARPA Subterranean Challenge: Multi-robotic Exploration of Underground Environments. In *Proceedings of the Modelling and Simulation for Autonomous Systems*; Mazal, J., Fagiolini, A., Vasik, P., Eds.; Springer: Cham, Switerland, 2020; pp. 274–290.
29. Tranzatto, M.; Dharmadhikari, M.; Bernreiter, L.; Camurri, M.; Khattak, S.; Mascarich, F.; Pfreundschuh, P.; Wisth, D.; Zimmermann, S.; Kulkarni, M.; et al. Team CERBERUS Wins the DARPA Subterranean Challenge: Technical Overview and Lessons Learned. *Field Robot.* **2024**, *4*, 349–312. [CrossRef]
30. Martínez-Rozas, S.; Rey, R.; Alejo, D.; Acedo, D.; Cobano, J.A.; Rodríguez-Ramos, A.; Campoy, P.; Merino, L.; Caballero, F. An Aerial/Ground Robot Team for Autonomous Firefighting in Urban GNSS-Denied Scenarios. *Field Robot.* **2022**, *2*, 241–273.
31. Chen, Q.; Irfan, M.; Yu, Q.; Sun, J.; Din, M.U.; Lin, D.; Dong, Y.; He, S.; Hussain, I. A Heterogeneous Multirobot System for Autonomous Object Retrieval in Challenging GNSS-Denied Maritime Environment. *J. Field Robot.* **2025**, *42*, 4288–4310. [CrossRef]

32. Baumann, C.; Martinoli, A. A modular functional framework for the design and evaluation of multi-robot navigation. *Robot. Auton. Syst.* **2021**, *144*, 103849. [CrossRef]
33. Testa, A.; Camisa, A.; Notarstefano, G. ChoiRbot: A ROS 2 toolbox for cooperative robotics. *IEEE Robot. Autom. Lett.* **2021**, *6*, 2714–2720. [CrossRef]
34. Lee, S.; Kim, T.; Chae, J.; Park, K.J. Optimizing ROS 2 Communication for Wireless Robotic Systems. *arXiv* **2025**, arXiv:2508.11366. [CrossRef]
35. Ohradzansky, M.T.; Rush, E.R.; Riley, D.G.; Mills, A.B.; Ahmad, S.; McGuire, S.; Biggie, H.; Harlow, K.; Miles, M.J.; Frew, E.W.; et al. Multi-Agent Autonomy: Advancements and Challenges in Subterranean Exploration. *Field Robot.* **2022**, *2*, 1068–1104. [CrossRef]
36. Caccavale, R.; Finzi, A. Plan Execution and Attentional Regulations for Flexible Human-Robot Interaction. In *Proceedings of the 2015 IEEE International Conference on Systems, Man, and Cybernetics*; IEEE: Piscataway, NJ, USA, 2015; pp. 2453–2458.
37. Caccavale, R.; Finzi, A. A Robotic Cognitive Control Framework for Collaborative Task Execution and Learning. *Top. Cogn. Sci.* **2022**, *14*, 327–343. [CrossRef]
38. Caccavale, R.; Finzi, A. Learning attentional regulations for structured tasks execution in robotic cognitive control. *Auton. Robot.* **2019**, *43*, 2229–2243. [CrossRef]
39. Labbe, M.; Michaud, F. RTAB-Map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation. *J. Field Robot.* **2019**, *36*, 416–446. [CrossRef]
40. Choset, H. Coverage of Known Spaces: The Boustrophedon Cellular Decomposition. *Auton. Robot.* **2000**, *9*, 247–253. [CrossRef]
41. Şucan, I.A.; Moll, M.; Kavraki, L.E. The Open Motion Planning Library. *IEEE Robot. Autom. Mag.* **2012**, *19*, 72–82. [CrossRef]
42. Pan, J.; Chitta, S.; Manocha, D. FCL: A general purpose library for collision and proximity queries. In *Proceedings of the 2012 IEEE International Conference on Robotics and Automation*; IEEE: Piscataway, NJ, USA, 2012; pp. 3859–3866.
43. MAVROS ROS Package. Available online: https://github.com/mavlink/mavros (accessed on 1 January 2026).
44. PX4 uXRCE-DDS Middleware Documentation. Available online: https://docs.px4.io/main/en/middleware/uxrce_dds (accessed on 1 January 2026).
45. Macenski, S.; Martin, F.; White, R.; Ginés Clavero, J. The Marathon 2: A Navigation System. In *Proceedings of the 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*; IEEE: Piscataway, NJ, USA, 2020.
46. Rösmann, C.; Hoffmann, F.; Bertram, T. Integrated online trajectory planning and optimization in distinctive topologies. *Robot. Auton. Syst.* **2017**, *88*, 142–153. [CrossRef]
47. Jocher, G.; Qiu, J. Ultralytics YOLO11. Available online: https://github.com/ultralytics/ultralytics (accessed on 10 November 2025).
48. QR Code Detection Dataset. Available online: https://universe.roboflow.com/qr-lmsul/qr-code-detection-jz2e3 (accessed on 10 November 2025).
49. ZBar ros2 Library. Available online: https://github.com/ros-drivers/zbar_ros (accessed on 10 November 2025).
50. Kalaitzakis, M.; Cain, B.; Carroll, S.; Ambrosi, A.; Whitehead, C.; Vitzilaios, N. Fiducial Markers for Pose Estimation. *J. Intell. Robot. Syst.* **2021**, *101*, 71. [CrossRef]
51. Sihite, E.; Kalantari, A.; Nemovi, R.; Ramezani, A.; Gharib, M. Multi-Modal Mobility Morphobot (M4) with appendage repurposing for locomotion plasticity enhancement. *Nat. Commun.* **2023**, *14*, 3323.
52. Gefen, E.; Zarrouk, D. Flying STAR2, a Hybrid Flying Driving Robot With a Clutch Mechanism and Energy Optimization Algorithm. *IEEE Access* **2022**, *10*, 115491–115502. [CrossRef]
53. Acar, O.; Honkavaara, E.; Botez, R.M.; Bayburt, D.C. Mechanisms and Control Strategies for Morphing Structures in Quadrotors: A Review and Future Prospects. *Drones* **2025**, *9*, 663. [CrossRef]
54. Zanasi, R.; Bianco, C.L.; Tonielli, A. Nonlinear filters for the generation of smooth trajectories. *Automatica* **2000**, *36*, 439–448. [CrossRef]
55. Morselli, R.; Zanasi, R.; Stramigioli, S. Discrete Second order trajectory generator with nonlinear constraints. *IFAC Proc. Vol.* **2005**, *38*, 200–205.
56. Biagiotti, L.; Melchiorri, C. Trajectory generation via FIR filters: A procedure for time-optimization under kinematic and frequency constraints. *Control Eng. Pract.* **2019**, *87*, 43–58. [CrossRef]
57. Zhang, Y.; Zhao, H.; Ye, C.; Ding, H. A Dynamic Real-Time Motion Planning Method for Multi-robots with Collision Avoidance. In *Proceedings of the Intelligent Robotics and Applications*; Huang, Y., Wu, H., Liu, H., Yin, Z., Eds.; Springer: Cham, Switerland, 2017; pp. 59–69.
58. Siciliano, B.; Sciavicco, L.; Villani, L.; Oriolo, G. *Robotics: Modelling, Planning and Control*; Springer: Cham, Switerland, 2009.


**Disclaimer/Publisher's Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.